Data Warehouses

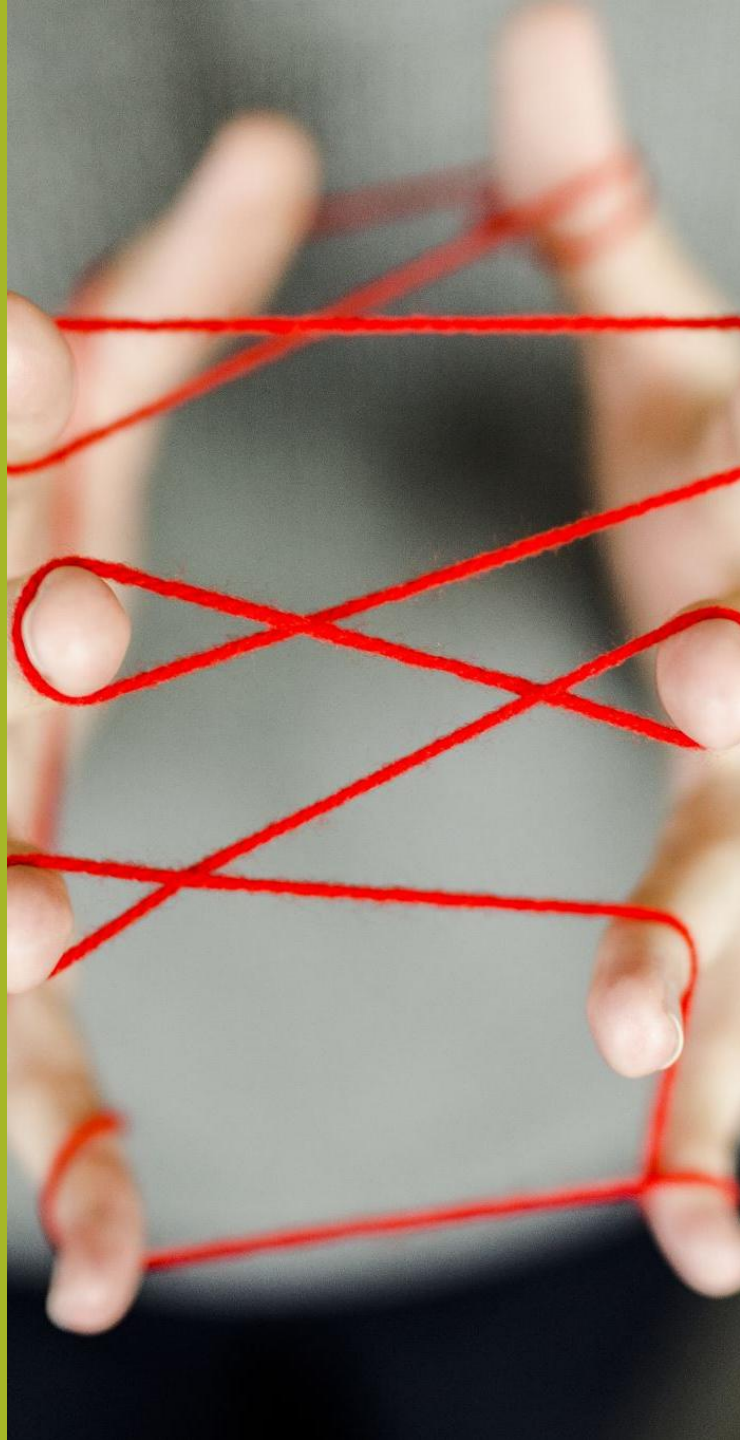
Lecture 13

Lecture Goals

Goal:

- Physical aspects of OLAP
  - *materialization*
  - *indexing,*
  - *partitioning,*
  - *columnar storage*

Physical aspects



What is Physical Design?

Physical design is the phase following logical design that identifies actual database tables and index structures used to implement the data warehouse.

This phase translates logical models into implementable database structures, making critical decisions that influence overall performance and maintenance.

Physical design decisions are primarily driven by performance optimization and operational efficiency requirements.

The physical design of the data warehouses is crucial to ensure adequate query response time

underlying data storage and processing solution

There are typically three common techniques for improving performance in data warehouse systems:

Materialized view

is a view that is physically stored in a database, which enhances query performance by precalculating costly operations such as joins and aggregations.

Indexing

alternative indexing mechanisms are used in data warehouses, typically bitmap and join indexes.

Partitioning or fragmentation

divides the contents of a relation into several files, typically based on a range of values of an attribute

Materialized views enhance query performance

by precalculating costly operations such as joins and aggregations

achieved at the expense of storage space.

Typical problems related to the use of materialized views:

Selection

select an appropriate set of views that minimizes the total query response time and the cost of maintaining the selected views

Calculation

Efficient creation of materialized views

Update

since all modifications to the underlying base tables must be propagated into the view (view maintenance)

Query rewrite

Trying to use the materialized views as much as possible, even if they only partially fulfill the query conditions

- Since data warehouse queries typically access a large number of tuples
  - *alternative indexing mechanisms are needed*
- Two common types of indexes for data warehouses are
  - *Bitmap indexes – useful for columns with a low number of distinct values (i.e., low cardinality attributes)*
    - several compression techniques can overcome low cardinality limitation
      - Bitslice Indices – use of bitmap indices by using bitmaps to index value ranges
  - *Join indexes – materialize a relational join between two tables by keeping pairs of row identifiers that participate in the join.*
    - In data warehouses, join indexes relate the values of dimensions to rows in the fact table.
      - Star index – join index between a fact table and each of its dimension tables
    - Join indexes can be combined with bitmap indexes - bitmap join indexes
      - Store pre-computed join results between fact and dimension tables - pre-materialized join results
- Other options
  - *aggregate index, summary index, hybrid b-tree/bitmap*

Partitioning or fragmentation is a mechanism frequently used in relational databases to reduce the execution time of queries.

It consists in dividing the contents of a relation into several files that can be more efficiently processed in this way.

There are two ways of partitioning a relation:

Vertical partitioning – splits the attributes of a table into groups that can be independently stored.

For example, a table can be partitioned such that the most often used attributes are stored in one partition, while other less often used attributes are kept in another partition.

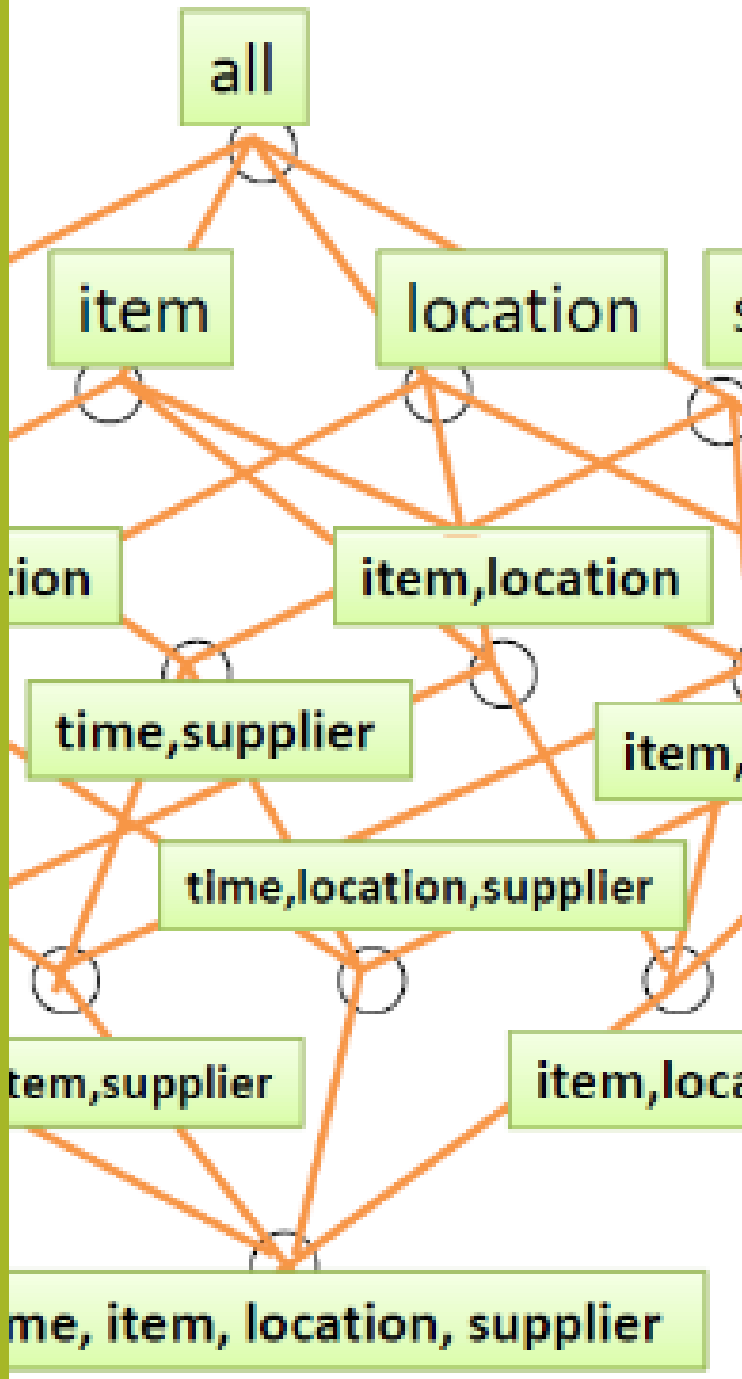
Horizontal partitioning – divides the records of a table into groups according to a particular criterion.

A common horizontal partitioning scheme in data warehouses is based on time, where each partition contains data about a particular time period, for instance, a year or a range of months.

Columnar storage

- *column store database is a type of database that stores data using a column-oriented model („stores all column data together”) – vertical partitioning*
  - Significantly increases performance for OLAP queries
  - Allows for better compression
- *has become the backbone of modern data warehousing due to its exceptional ability to handle analytical workloads efficiently.*
  - Data warehouses primarily serve analytical workloads that scan large numbers of rows while typically touching only a small subset of columns
  - Unlike traditional row-based storage systems, columnar storage organizes data by columns rather than rows, creating significant advantages for the types of queries commonly performed in data warehouse environments
    - In a columnar data warehouse, each column of a table is stored separately as a contiguous block of data on disk
  - Benefits are substantial
    - Using columnar storage, each data block can hold column field values for as many as three times the number of records compared to row-based storage, reducing I/O operations by approximately two-thirds

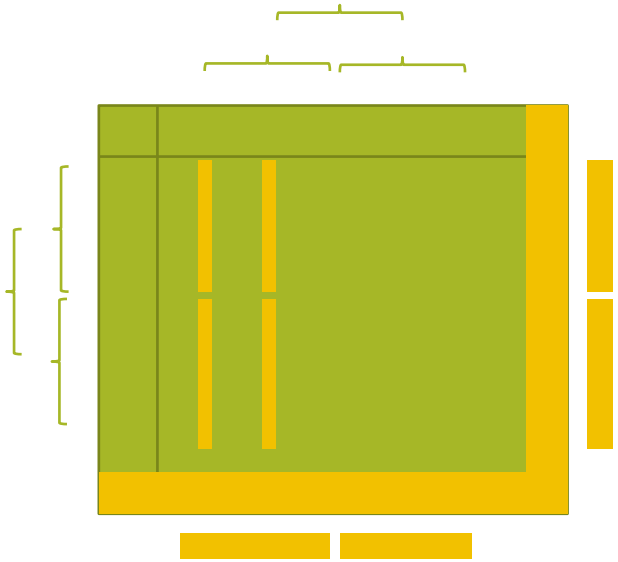
Materialization



Aggregates

In multidimensional data model together with measure values usually we store summarizing information

aggregates



Location: Vancouver					
Time (quarters)	Items				
	TV	Computer	Phone	Security	Sum
Q1	605	825	14	400	1844
Q2	680	952	31	512	2175
Q3	812	1023	30	501	2366
Q4	927	1038	38	580	2583
Sum	3024	3838	113	1993	8968

Materialized views

Materialized views that hold aggregates

- are one of the most important means of improving query performance in a multidimensional database management system

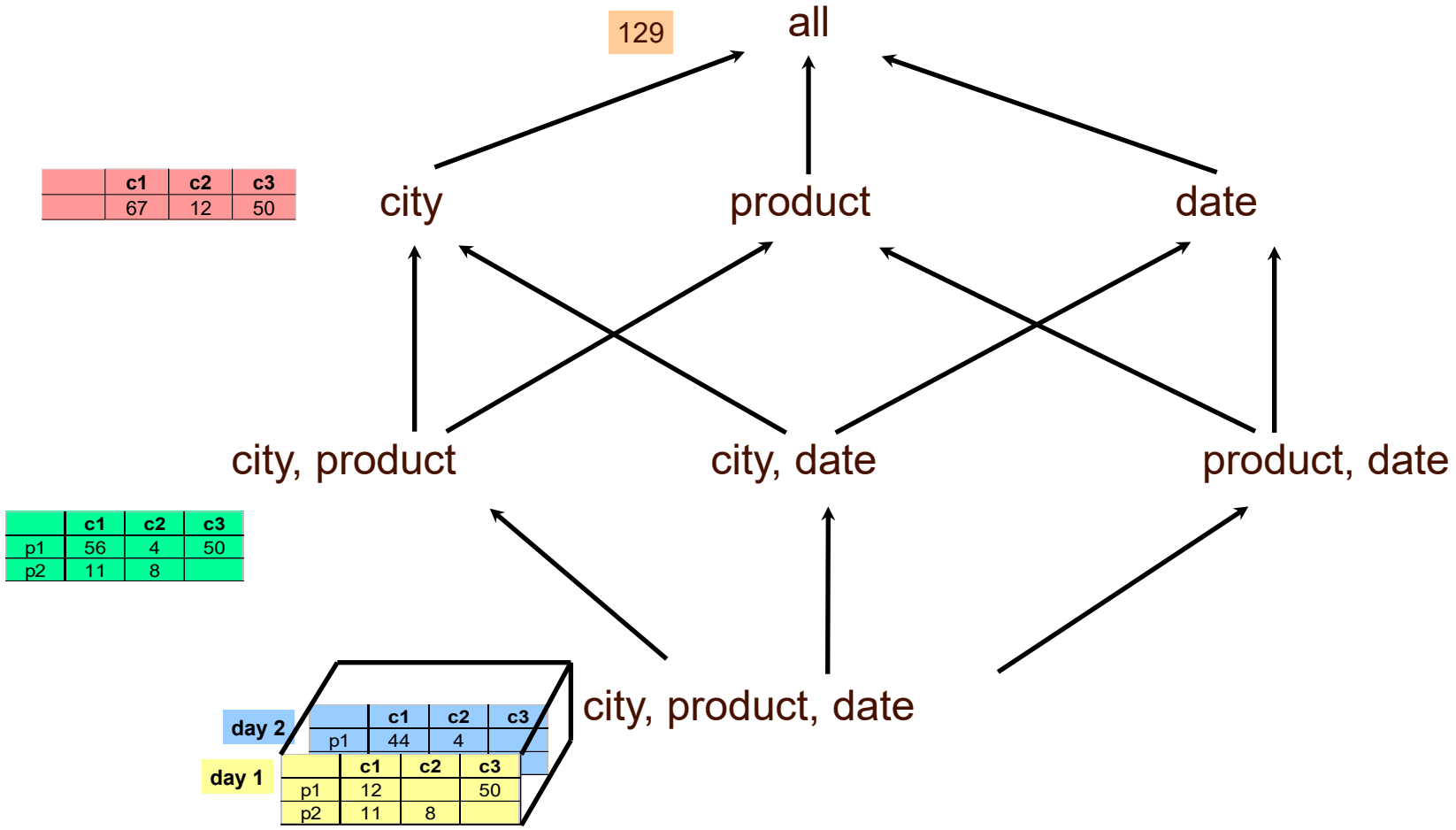
Precomputed aggregate view is the materialized result of a query

- that aggregates (e.g., summarizes) parts of or all the data in the database

The word "materialized" means that the result is stored physically

- the system can answer the query by just reading the (already computed and stored) result
- instead of computing it from the detailed base data

Cube Aggregates Lattice



	c1	c2	c3
	67	12	50

	c1	c2	c3
p1	56	4	50
p2	11	8	

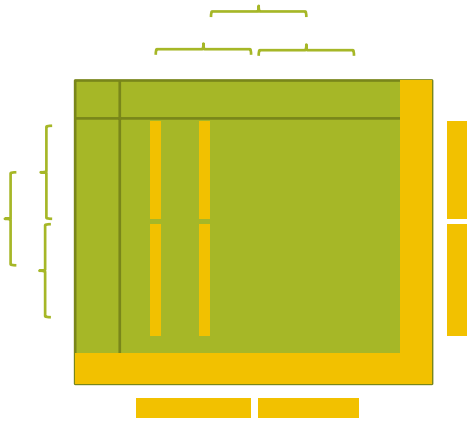
	c1	c2	c3
p1	44	4	

	c1	c2	c3
p1	12		50
p2	11	8	

Aggregates

In multidimensional data model together with measure values we store summarizing information

- aggregates



Consider the following 2 cases for n-dimensional cube

- Case 1: Dimensions have no hierarchies
  - *Then the total number of cuboids computed for a n-dimensional cube*

$$2^n$$

- Case 2: Dimensions have hierarchies
  - *Then the total number of cuboids computed for a n-dimensional cube*

$$\prod_{i=1}^{n}(L_i + 1)$$

where Li is the number of levels associated with dimension i; n is the number of dimensions

Materializing cubes

Example

Cube with 3 dimensions – item, city, year

2^3 cuboids

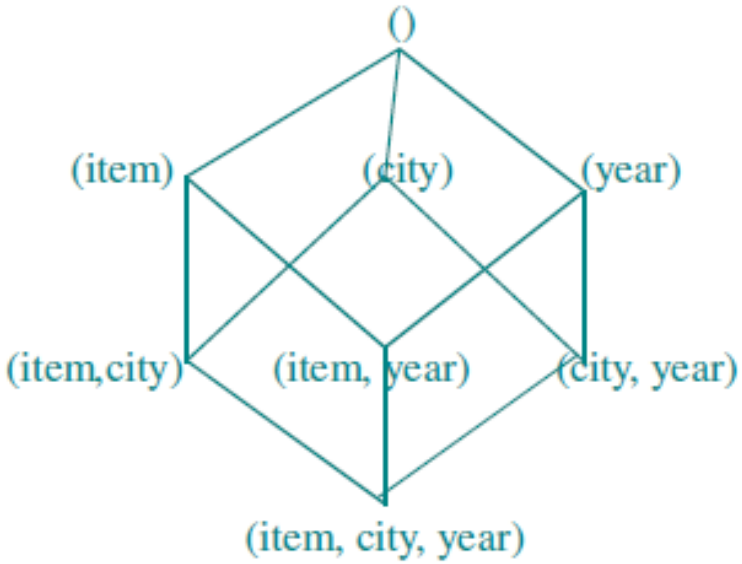
Need compute the following groupings (group by)

(item, city, year),

(item, city),(item, year), (city, year),

(item), (city), (year)

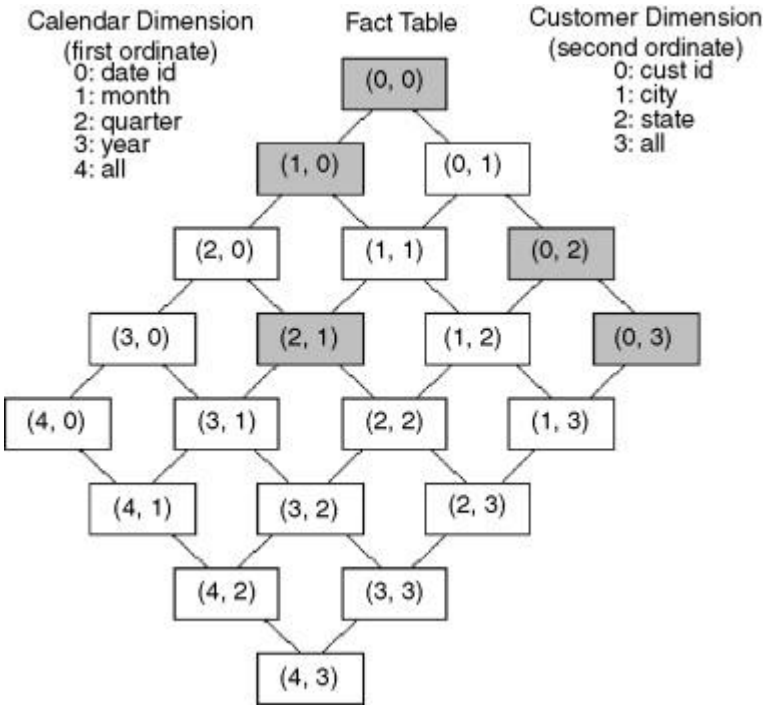
()



Materializing cubes

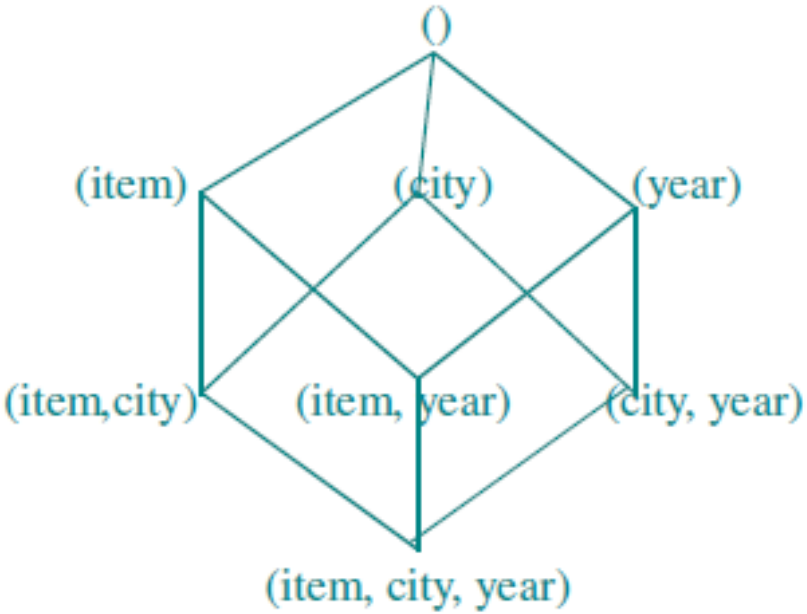
Example

- Cube with 2 dimensions
  - *One dimension with 4 levels*
  - *One dimension with 3 levels*
  - *In total:*
    - 5*4=20



Materializing cubes

Why cuboids are important?



Processing

Materialized views

The use of materialized views can speed up query processing

to be several hundreds or thousands times faster than when using the base data.

Often many different queries can even be answered efficiently by using the same materialized view.

If we, for example, have an aggregate that holds the summarized sales for cities,

we can also use that aggregate to calculate the total sales for states and the single total of all sales.

Transparent to the end user

should not be burdened with the task of deciding when to use a precomputed aggregate

OLAP system must take care of this transparently to the user

Materializing cubes

View materialization in data warehouses presents a fundamental trade-off between three competing factors:

query processing cost,

view maintenance cost,

and storage space requirements

The goal is to select an appropriate set of views

that minimizes the sum of query response time and maintenance costs while operating within resource constraints such as storage space and maintenance time window

Materializing cubes

The problem is then to decide which precomputed aggregates to create.

- *One extreme is to not have any.*
  - But if there are no precomputed aggregates, query responses will be slow.
- *The other extreme is to precompute each possible aggregate.*
  - Unfortunately, they will occupy huge amounts of disk space and will take unreasonably long to compute in the first place.
  - Further, they need to be updated when updates to the base data occur.
  - So this extreme is also not practical.
- *A good compromise in-between the two extremes must therefore be found.*
  - Unfortunately, the problem of picking the optimal set of aggregates is an NP hard problem.

Materializing cubes

The problem lies in selection of cuboids to be materialized

high number of materialized cuboids

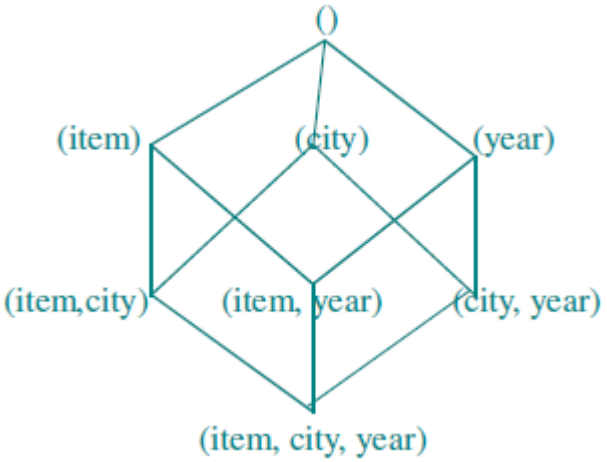
huge size of data warehouse.

small number of materialized cuboids

slow query processing

use information to balance the two

like size and storage constraints, dependency, update and maintenance costs, access frequency and patterns, cost (mixture of the former)



Materializing cubes

Materialization of data cube

Materialize every cuboid (full materialization)

full cube

all the cells of all of the cuboids for a given data cube

Materialize none (no materialization)

base cube

minimal set of cells cube

Materialize some (partial materialization)

partial cube

subset of full cube

Partial

Partial cube materialization

Selected parts of a data cube (selected cuboids) are materialized

Hybrid approach

- Balance between the response time and required storage space and overhead
  - Instead of computing the full cube, we can compute only a subset of the data cube's cuboids, or subcubes consisting of subsets of cells from the various cuboids.

Requires

- to identify a subset of cuboids that will be materialized
  - Many cells in a cuboid may actually be of little or no interest to the data analyst
- efficient update during each data load and refresh

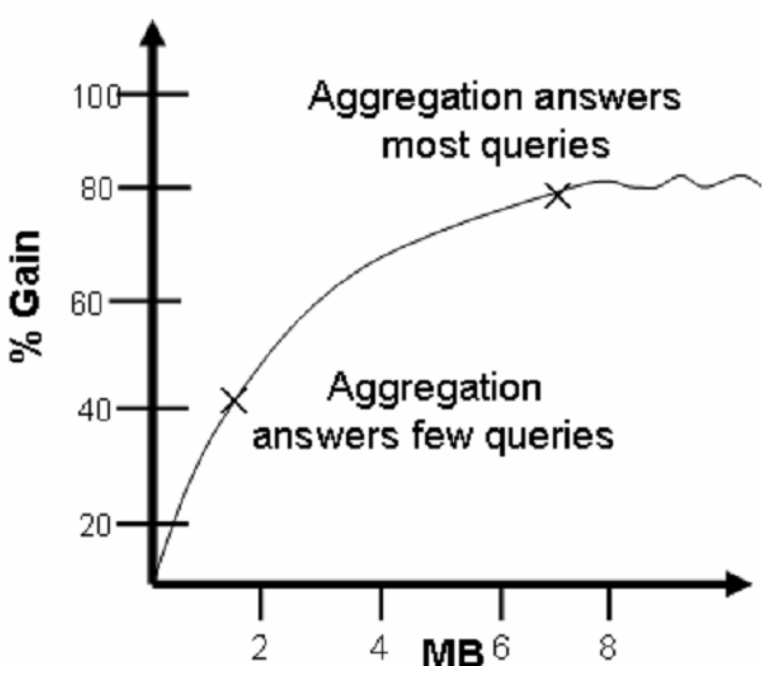
We can use some metadata

- Usage profiles, Sparsity information, Etc.

Partial

Performance vs. Space Trade –off

- Maximum performance boost
  - *implies using lots of disk space for storing every pre calculation.*
- Minimum performance boost
  - *implies no disk space with zero pre - calculation.*
- Using meta-data/statistics
  - *to determine best level of pre-aggregation from which all other aggregates can be computed.*



Materializing cubes

A typical problem of materialized views is

updating

since all modifications to the underlying base tables must be propagated into the view (view maintenance)

selection of materialized views

select an appropriate set of views that minimizes the total query response time and the cost of maintaining the selected views

query rewriting

tries to use the materialized views as much as possible, even if they only partially fulfill the query conditions

Materialized View Selection

Several OLAP products have adopted heuristic approaches for cuboid and subcube selection.

- *A popular approach is to materialize the set of cuboids on which other frequently referenced cuboids are based.*
  - e.g. Greedy Lattice-Based Selection (The HRU Algorithm)
  - Constraint-Based / Resource-Aware Materialization
- *Alternatively, we can compute an* ***iceberg cube***
  - data cube that stores only those cube cells whose aggregate value (e.g., count) is above some minimum support threshold.
- *Another common strategy is to materialize a* ***shell cube****.*
  - This involves precomputing the cuboids for only a small number of dimensions (such as 3 to 5) of a data cube.
  - The dataset's dimensions are partitioned into disjoint groups and each group, called a **shell fragment**, is processed independently - ***full local data cube*** *is computed for each fragment*
    - *retains inverted indices mapping attribute values to tuple IDs*
- *Use usage information – Workload-Directed / Query-Directed Selection*

Materializing cubes

Effective view materialization relies on comprehensive cost models

that evaluate the total expenses associated with each materialization decision

These models typically incorporate:

Query Processing Costs:

The computational expense of executing queries against the selected set of materialized views, considering factors such as join operations, aggregations, and data scanning requirements.

Maintenance Costs:

The overhead associated with keeping materialized views synchronized with their underlying base tables, including the time and resources required for incremental or complete refreshes.

Storage Costs:

The disk space requirements for storing materialized views, which directly impacts both hardware costs and system performance.

Warehouse Maintenance

- Warehouse data ≈ materialized view
  - *Initial loading*
  - *View maintenance*
- Derived Warehouse Data
  - *indexes*
  - *aggregates*
  - *materialized views*
- View maintenance

Choosing view

Query rewriting

Materialized views

Efficient Processing of OLAP Queries

The purpose of materializing cuboids and constructing OLAP index structures is to speed up query processing in data cubes.

Given materialized views, query processing should proceed as follows:

1. Determine which operations should be performed on the available cuboids

This involves transforming any selection, projection, roll-up (group-by), and drill-down operations specified in the query into corresponding SQL and/or OLAP operations.

For example, slicing and dicing a data cube may correspond to selection and/or projection operations on a materialized cuboid.

2. Determine to which materialized cuboid(s) the relevant operations should be applied

Choose view to answer a query

- Identify all the materialized cuboids that may potentially be used to answer a query.
- Pruning the above set using knowledge of dominance relationship among cuboids.
- Estimating the costs of using the remaining materialized cuboids and selecting the cuboid with the least cost.

Example

- Cube
  - *Fact Table - sales [time, item, location]:*
    - sum(sales_in_dollars)
  - *Dimension's hierarchies used are :*
    - day<month<quarter<year for time
    - item_name<brand<type for item
    - street<city<province_or_state<country for location
- Cuboids are :
  - *cuboid 1: {item, city, year}*
  - *cuboid 2: {brand, country, year}*
  - *cuboid 3: {brand, state, year}*
  - *cuboid 4: {item, state} where year = 2000*
- Query
  - *{brand, state} with year = 2000?*

Analysis

- cuboid 1
  - *costs the most since item and city are at lower level than brand and state*
- cuboid 2
  - *can not be used since state < country*
- cuboid 3
  - *is better than cuboid 4 if there are few year values associated with items and several items for each brand*
- cuboid 4
  - *is better than cuboid 3 if efficient indices are available for cuboid 4.*

Example

- cuboid 1:
  - *{item, city, year}*
- cuboid 2:
  - *{brand, country, year}*
- cuboid 3:
  - *{brand, state, year}*
- cuboid 4:
  - *{item, state} where year = 2000*

Query

{brand, state} with year = 2000?

What is the optimal set of views to materialize?

How many impressions do we need to materialize to get reasonable performance?

(part, supplier)

```
SELECT part, supplier, sum(sales)
FROM Sales
GROUP BY part, supplier
```

(part)

```
SELECT part, sum(sales)
FROM Sales
GROUP BY part
```

(supplier)

```
SELECT supplier, sum(sales)
FROM Sales
GROUP BY supplier
```

(part, customer)

```
SELECT part, customer, sum(sales)
FROM Sales
GROUP BY part, customer
```

Query	Answer	Cost
• (part,supplier,customer)	6M	6M
• (part,customer)	6M	6M
• (part,supplier)	0.8M	6M
• (supplier,customer)	6M	6M
• (part)	0.2M	6M
• (supplier)	0.01M	6M
• (customer)	0.1M	6M
• ()	1	6M

(part, supplier)

```
SELECT part, supplier, sum(sales)
FROM Sales
GROUP BY part, supplier
```

(part)

```
SELECT part, sum(sales)
FROM Sales
GROUP BY part
```

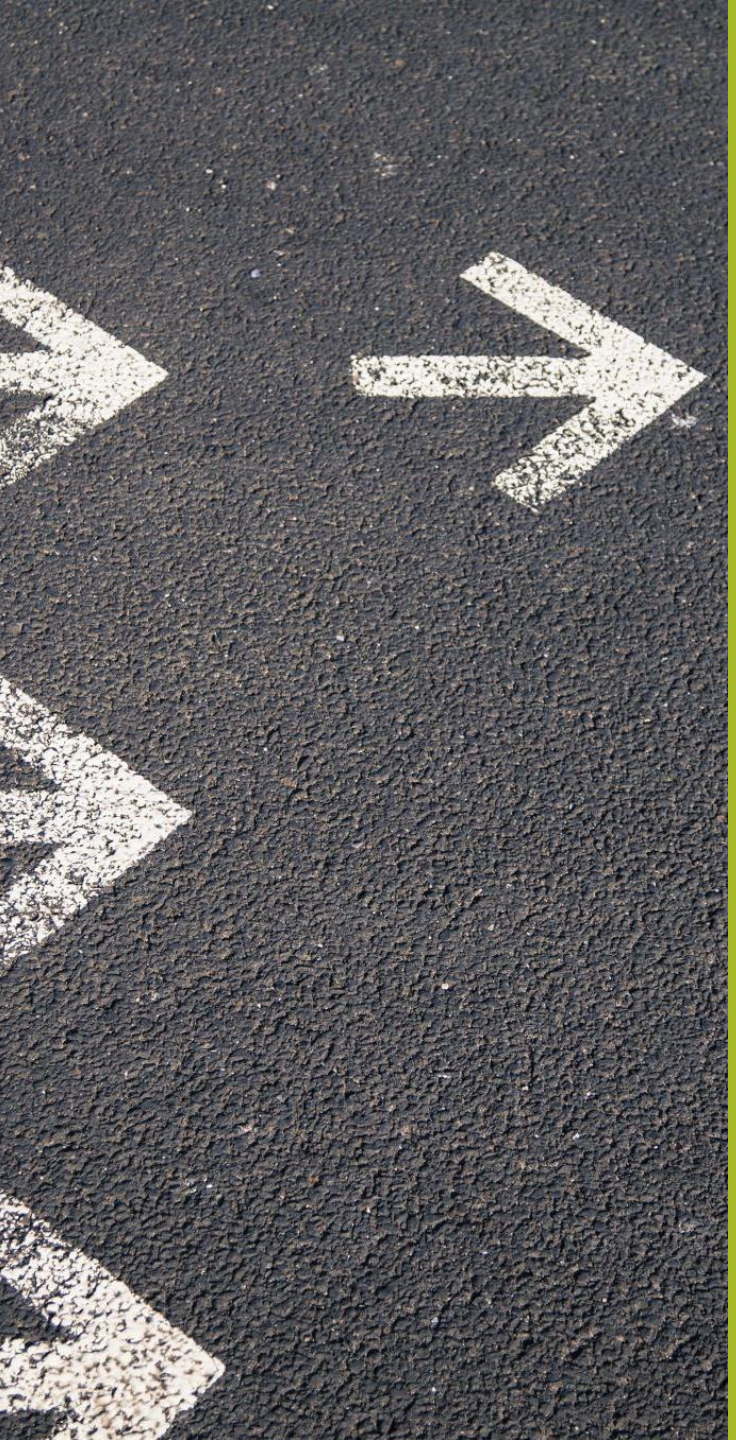
(supplier)

```
SELECT supplier, sum(sales)
FROM Sales
GROUP BY supplier
```

(part, customer)

```
SELECT part, customer, sum(sales)
FROM Sales
GROUP BY part, customer
```

Query	Answer	Cost
(part,supplier,customer)	6M	6M
(part,customer)	6M	6M
(part,supplier)	0.8M	0.8M
(supplier,customer)	6M	6M
(part)	0.2M	0.8M
(supplier)	0.01M	0.8M
(customer)	0.1M	0.1M
()	1	0.1M



Indexing in DW

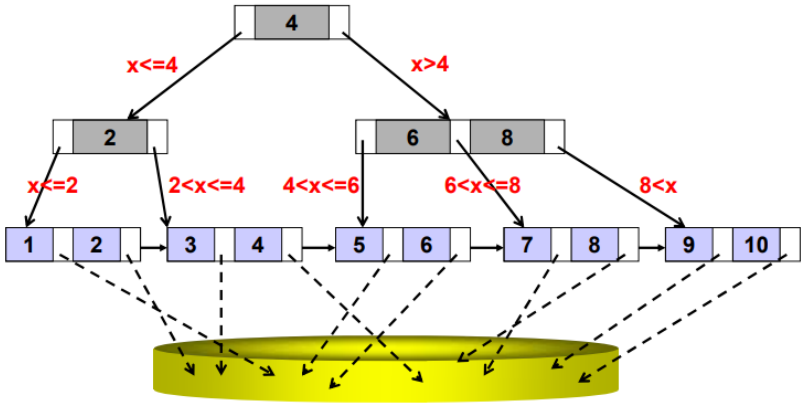
Indexing is another important means of improving query performance

- ***Fact tables*** *are typically very large and are used in the majority of queries.*
  - Therefore, a fact table often has many indices such that most queries can benefit from one or more of them
  - Typically, a separate index is built on each dimension key
    - If the DBMS supports index intersection, the indices can then be used in combination when answering a query
  - It can also be a good idea, however, to create indices on combinations of dimension keys for those combinations that are often used together by queries
  - In general, the key referencing the Date dimension should be put first in combinations since most queries refer to dates
  - Common approach:
    - Clustered / Sort Keys (On Date/Time)
    - Zone Maps / Data Skipping (On Sort Keys and Measures)
    - Bitmap Indexes (On Foreign Keys)

Indexing is another important means of improving query performance

- ***Dimension tables*** *should also be indexed.*
  - Depending on typical usage, it may or may not make sense to build indices on all individual columns.
  - As for fact tables, indices can be built on combinations of attributes.
  - Common approach:
    - Standard B-Tree Indexes (On Primary Keys)
    - Bitmap Indexes (On Descriptive Columns)

B+Tree



B-tree stores data such that each node contains keys in ascending order.

Each of these keys has two references to another two child nodes.

The left-side child node keys are less than the current keys,

and the right-side child node keys are more than the current keys.

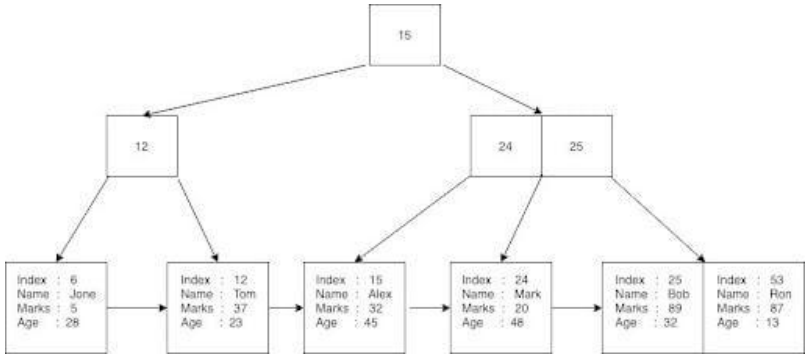
If a single node has “n” number of keys

then it can have maximum “n+1” child nodes.

B+tree stores data on the leaf nodes

This means that all non-leaf node values are duplicated in leaf nodes again.

Clustered

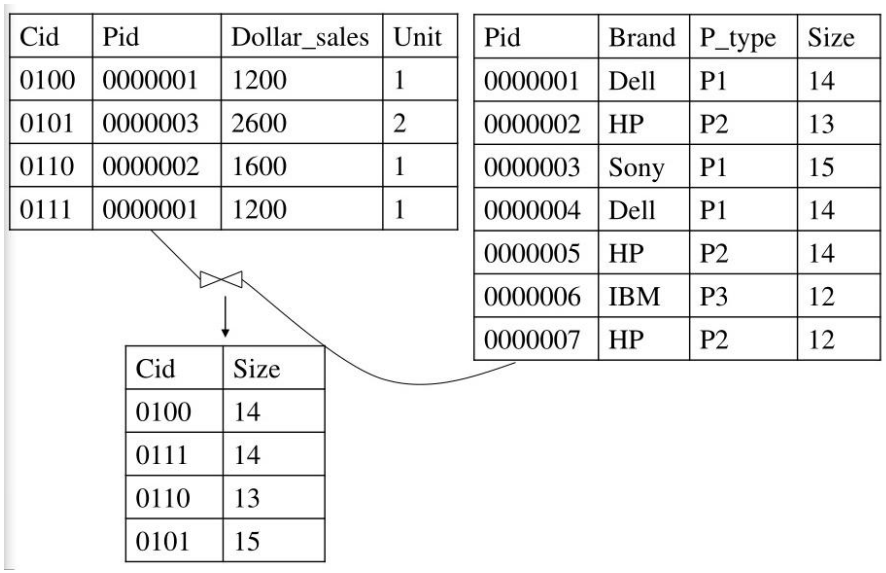


Sr. No.	Operation	Time Complexity
1.	Search	O(log n)
2.	Insert	O(log n)
3.	Delete	O(log n)
4.	Traverse	O(n)

Note: "n" is the total number of elements in the B-tree

https://planetscale.com/blog/btrees-and-database-indexes

Join index



Cid	Pid	Dollar_sales	Unit
0100	0000001	1200	1
0101	0000003	2600	2
0110	0000002	1600	1
0111	0000001	1200	1

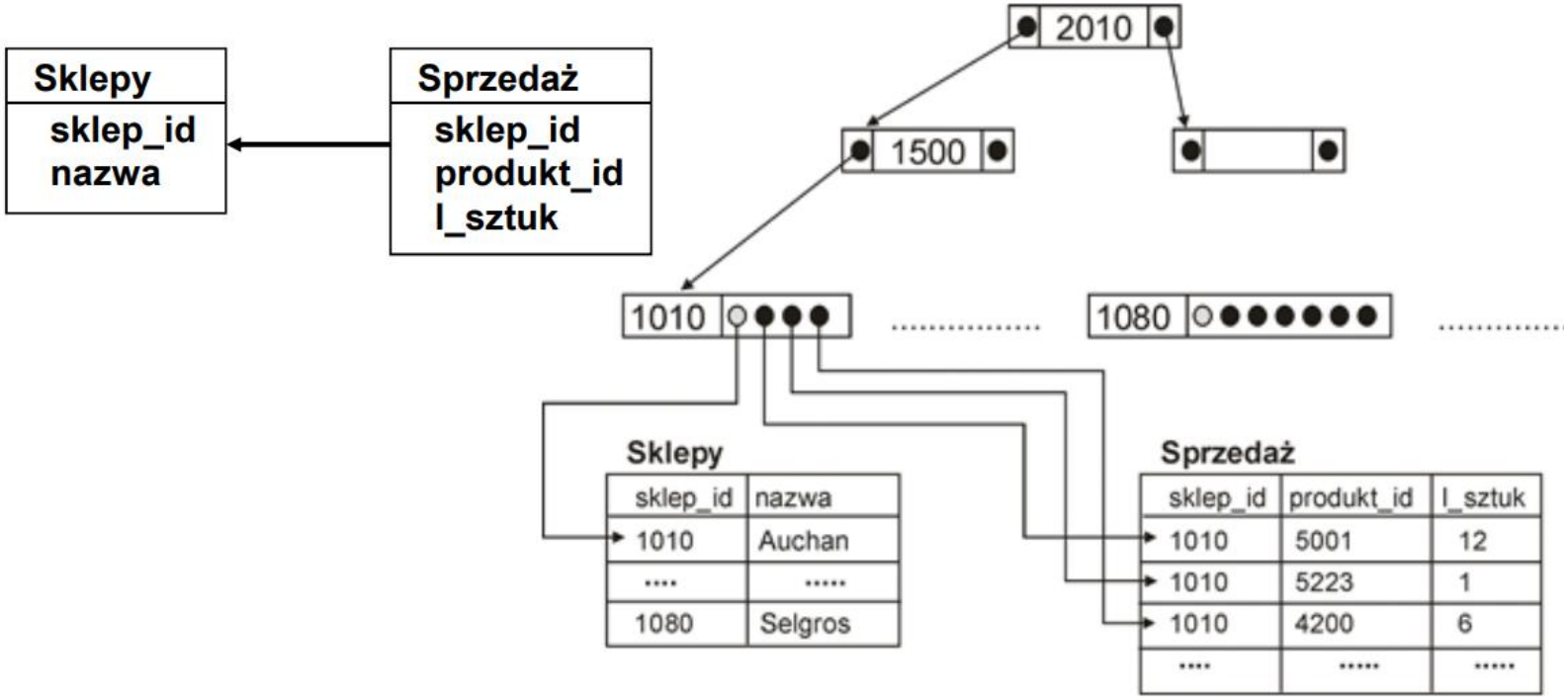
Pid	Brand	P_type	Size
0000001	Dell	P1	14
0000002	HP	P2	13
0000003	Sony	P1	15
0000004	Dell	P1	14
0000005	HP	P2	14
0000006	IBM	P3	12
0000007	HP	P2	12

Cid	Size
0100	14
0111	14
0110	13
0101	15

A join index:

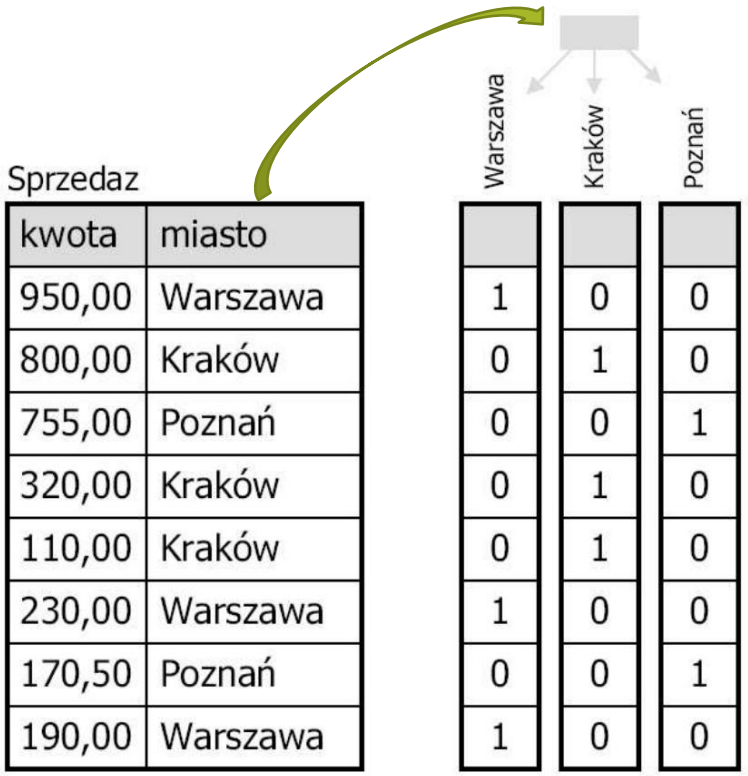
an index on one table that involves a column value from different table through a commonly encountered join.

np. indeks na atrybucie Sklepy.sklep_id



Bitmaps

- The principle of the bitmap index is that the position bitmap for the value v
  - *has a 1 on position p*
    - if and only if the p'th row in the table takes the value v
  - *has a 0*
    - otherwise
- A bitmap index makes it very fast to locate rows with a certain value
  - *Just identify 1s*
- It is also possible to combine bitmap indices
  - *Multiple indices is just computing the logical AND or OR or XOR*
- It is also possible to use other operators on bitmap indices
  - *Negation*



Sprzedaz

kwota	miasto	Warszawa	Kraków	Poznań
950,00	Warszawa	1	0	0
800,00	Kraków	0	1	0
755,00	Poznań	0	0	1
320,00	Kraków	0	1	0
110,00	Kraków	0	1	0
230,00	Warszawa	1	0	0
170,50	Poznań	0	0	1
190,00	Warszawa	1	0	0

Using Bitmaps

Query:

Get people with age = 20 and name = "fred"

Result:

Bit Map for age = 20:

1 1 0 1 1 0 0 0

Bit Map for name = "fred":

0 1 0 0 0 0 0 0

Answer is intersection:

_ 1 _ _ _ _ _ _

rId	name	age
r4	joe	20
r18	fred	20
r19	sally	21
r34	nancy	20
r35	tom	20
r36	pat	25
r5	dave	21
r41	jeff	26

Bitmap Index

Index on a particular column

- *Each value in the column has a bit vector: bit-op is fast*
- *The length of the bit vector: # of records in the base table*
- *The i-th bit is set if the i-th row of the base table has the value for the indexed column*
- *not suitable for high cardinality domains*

Base table

Cust	Region	Type
C1	Asia	Retail
C2	Europe	Dealer
C3	Asia	Dealer
C4	America	Retail
C5	Europe	Dealer

Index on Region

RecID	Asia	Europe	America
1	1	0	0
2	0	1	0
3	1	0	0
4	0	0	1
5	0	1	0

Index on Type

RecID	Retail	Dealer
1	1	0
2	0	1
3	0	1
4	1	0
5	0	1

Bitmap index

a

ProductKey	ProductName	QuantityPerUnit	UnitPrice	Discontinued	CategoryKey
p1	prod1	25	60	No	c1
p2	prod2	45	60	Yes	c1
p3	prod3	50	75	No	c2
p4	prod4	50	100	Yes	c2
p5	prod5	50	120	No	c3
p6	prod6	70	110	Yes	c4

b

	25	45	50	70
p1	1	0	0	0
p2	0	1	0	0
p3	0	0	1	0
p4	0	0	1	0
p5	0	0	1	0
p6	0	0	0	1

c

	60	75	100	110	120
p1	1	0	0	0	0
p2	1	0	0	0	0
p3	0	1	0	0	0
p4	0	0	1	0	0
p5	0	0	0	0	1
p6	0	0	0	1	0

Bitmap join index

In addition to a bitmap index on a single table, you can create a bitmap join index

a bitmap index for the join of two or more tables.

Bitmap join index

the bitmap for the table to be indexed is built for values coming from the joined tables.

In a data warehousing environment, the join condition is an equi-inner join between the primary key column or columns of the dimension tables and the foreign key column or columns in the fact table.

can improve the performance by an order of magnitude.

By storing the result of a join, the join can be avoided completely for SQL statements using a bitmap join index.

Bitmap join index

RowID Product	Product Key	Product Name	...	Discontinued	...
1	p1	prod1	...	No	...
2	p2	prod2	...	Yes	...
3	p3	prod3	...	No	...
4	p4	prod4	...	Yes	...
5	p5	prod5	...	No	...
6	p6	prod6	...	Yes	...

RowID Sales	Product Key	Customer Key	Time Key	Sales Amount
1	p1	c1	t1	100
2	p1	c2	t1	100
3	p2	c2	t2	100
4	p2	c2	t3	100
5	p3	c3	t3	100
6	p4	c3	t4	100
7	p5	c4	t5	100

join index

RowID Sales	RowID Product
1	1
2	1
3	2
4	2
5	3
6	4
7	5

join bitmap index

Yes	No
0	1
0	1
1	0
1	0
0	1
1	0
0	1

Bitmap join index

RowID Product	Product Key	Product Name	...	Discontinued	...
1	p1	prod1	...	No	...
2	p2	prod2	...	Yes	...
3	p3	prod3	...	No	...
4	p4	prod4	...	Yes	...
5	p5	prod5	...	No	...
6	p6	prod6	...	Yes	...

Customer Key	Customer Name	Address	Postal Code	...
c1	cust1	35 Main St.	7373	...
c2	cust2	Av. Roosevelt 50	1050	...
c3	cust3	Av. Louise 233	1080	...
c4	cust4	Rue Gabrielle	1180	...

Star queries

Require join of fact and dimension tables

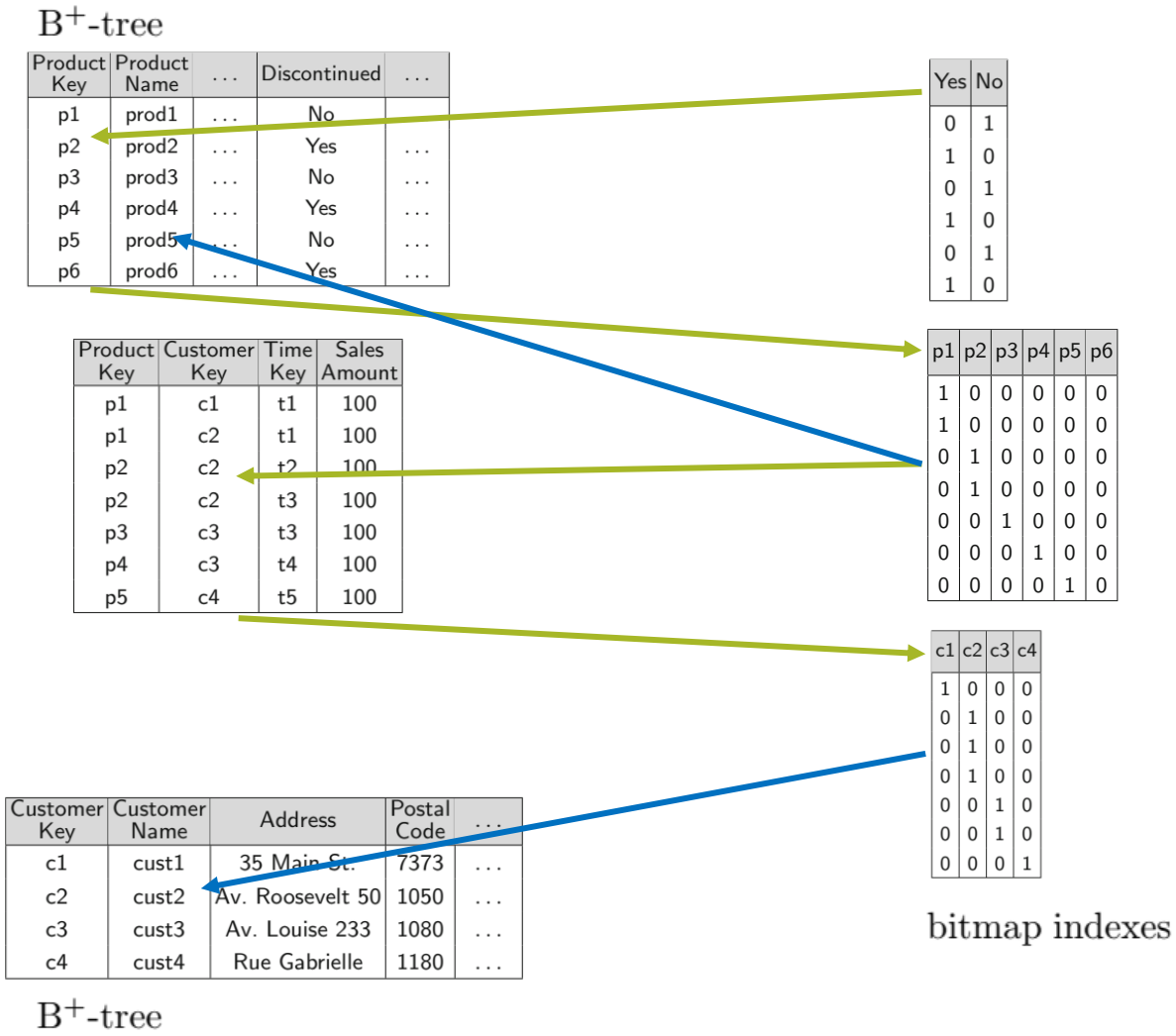
Example

Total sales of discontinued products by client and product:

```
SELECT    C.CustomerName, P.ProductName, SUM(S.SalesAmount)
FROM      Sales S, Customer C, Product P
WHERE     S.CustomerKey = C.CustomerKey AND
          S.ProductKey = P.ProductKey AND P.Discontinued = 'Yes'
GROUP BY C.CustomerName, P.ProductName
```

How is this query handled by the system?

RowID Sales	Product Key	Customer Key	Time Key	Sales Amount
1	p1	c1	t1	100
2	p1	c2	t1	100
3	p2	c2	t2	100
4	p2	c2	t3	100
5	p3	c3	t3	100
6	p4	c3	t4	100
7	p5	c4	t5	100


B+-tree
Product Key
Product Name
...
Discontinued
...
p1 prod1 ... No
p2 prod2 ... Yes ...
p3 prod3 ... No ...
p4 prod4 ... Yes ...
p5 prod5 ... No ...
p6 prod6 ... Yes ...
Yes No
0 1
1 0
0 1
1 0
0 1
1 0
Product Key
Customer Key
Time Key
Sales Amount
p1 c1 t1 100
p1 c2 t1 100
p2 c2 t2 100
p2 c2 t3 100
p3 c3 t3 100
p4 c3 t4 100
p5 c4 t5 100
p1 p2 p3 p4 p5 p6
1 0 0 0 0 0
1 0 0 0 0 0
0 1 0 0 0 0
0 1 0 0 0 0
0 0 1 0 0 0
0 0 0 1 0 0
0 0 0 0 1 0
c1 c2 c3 c4
1 0 0 0
0 1 0 0
0 1 0 0
0 1 0 0
0 0 1 0
0 0 1 0
0 0 0 1
bitmap indexes
Customer Key
Customer Name
Address
Postal Code
...
c1 cust1 35 Main St. 7373 ...
c2 cust2 Av. Roosevelt 50 1050 ...
c3 cust3 Av. Louise 233 1080 ...
c4 cust4 Rue Gabrielle 1180 ...
B+-tree

Product Key	Product Name	...	Discontinued	...
p1	prod1	...	No	...
p2	prod2	...	Yes	...
p3	prod3	...	No	...
p4	prod4	...	Yes	...
p5	prod5	...	No	...
p6	prod6	...	Yes	...

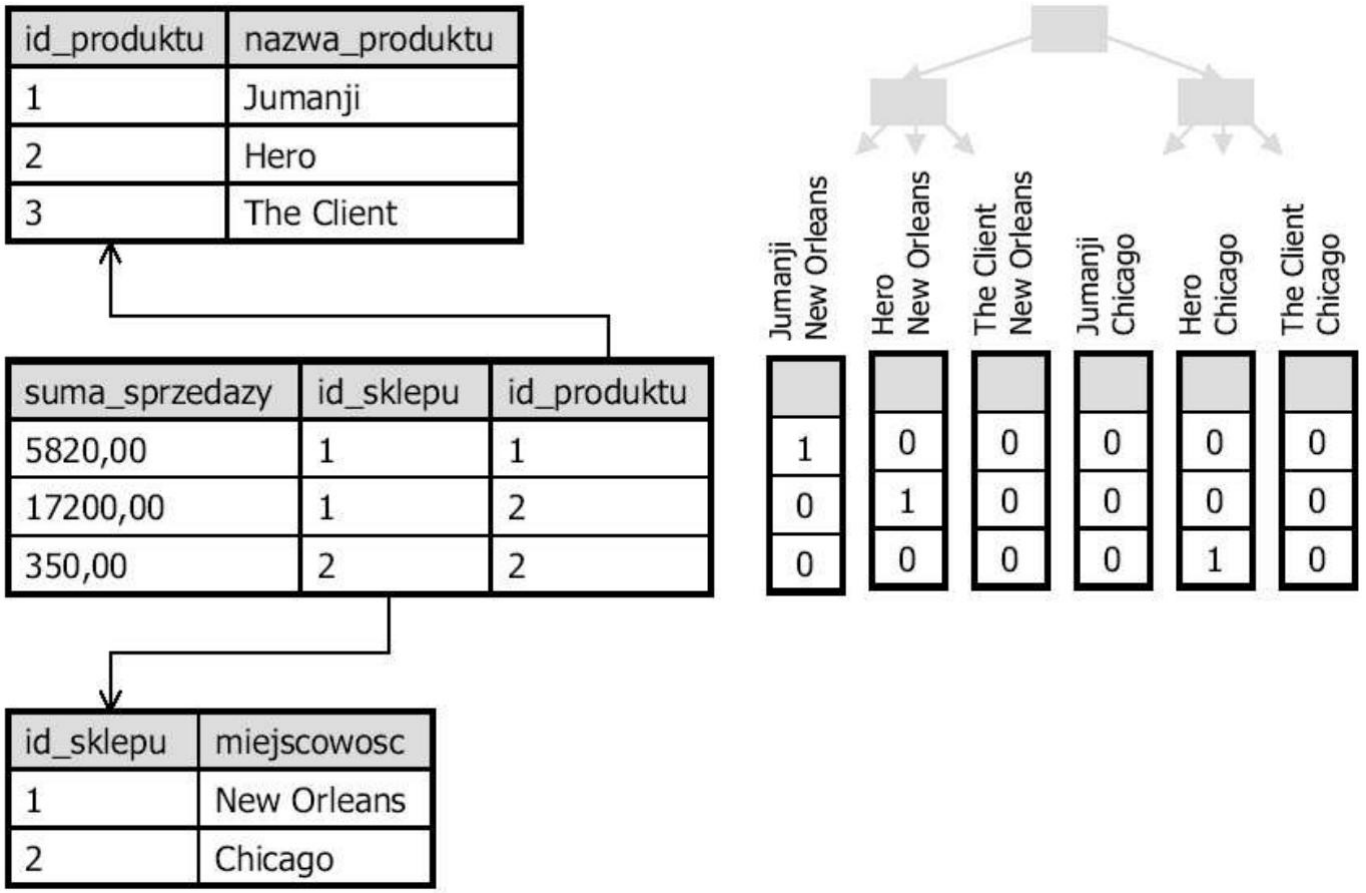
Product Key	Customer Key	Time Key	Sales Amount
p1	c1	t1	100
p1	c2	t1	100
p2	c2	t2	100
p2	c2	t3	100
p3	c3	t3	100
p4	c3	t4	100
p5	c4	t5	100

Yes	No
0	1
0	1
1	0
1	0
0	1
1	0
0	1

bitmap join indexes

Customer Key	Customer Name	Address	Postal Code	...
c1	cust1	35 Main St.	7373	...
c2	cust2	Av. Roosevelt 50	1050	...
c3	cust3	Av. Louise 233	1080	...
c4	cust4	Rue Gabrielle	1180	...

Bitmap join index



id_produktu	nazwa_produktu
1	Jumanji
2	Hero
3	The Client

suma_sprzedazy	id_sklepu	id_produktu
5820,00	1	1
17200,00	1	2
350,00	2	2

id_sklepu	miejscowosc
1	New Orleans
2	Chicago

Columnar storage

Columnar storage

We store the database in storage.

Data is written to storage in contiguous chunks.

In row oriented databases

these chunks contain a row consisting of all of its column values as a tuple.

In column oriented databases

these chunks contain only the values in each row that belongs to that column.

Given a certain amount of space on an SSD or HDD, a chunk of rows/columns data is read at once.

This difference has a significant impact on performance.

If the data you need to access is stored mostly in a small number of columns and it is not necessary to query each field in the rows

you may be better off with a columnar-store

If you need many columns in each row to determine which rows are relevant

a row-store may be a better fit.

Columnar storage

Example

Retrieving all columns from row 1 through row 5 requires a relatively low effort as the data is contiguous and ordered

Row 1-1	Row 1-2	Row 1-3	Row 1-4	Row 1-5	Row 2-1	Row 2-2	Row 2-3	Row 2-4
Row 2-5	Row 3-1	Row 3-2	Row 3-3	Row 3-4	Row 3-5	Row 4-1	Row 4-2	Row 4-3
Row 4-4	Row 4-5	Row 5-1	Row 5-2	Row 5-3	Row 5-4	Row 5-5	Row 6-1	Row 6-2
Row 6-3	Row 6-4	Row 6-5	Row 7-1	Row 7-2	Row 7-3	Row 7-4	Row 7-5	Row 8-1
Row 8-2	Row 8-3	Row 8-4	Row 8-5	Row 9-1	Row 9-2	Row 9-3	Row 9-4	Row 9-5

Consider a typical OLAP query against the example table where a single column is aggregated across a large portion of the table that happens to include all of the rows shown here.

Row 1-1	Row 1-2	Row 1-3	Row 1-4	Row 1-5	Row 2-1	Row 2-2	Row 2-3	Row 2-4
Row 2-5	Row 3-1	Row 3-2	Row 3-3	Row 3-4	Row 3-5	Row 4-1	Row 4-2	Row 4-3
Row 4-4	Row 4-5	Row 5-1	Row 5-2	Row 5-3	Row 5-4	Row 5-5	Row 6-1	Row 6-2
Row 6-3	Row 6-4	Row 6-5	Row 7-1	Row 7-2	Row 7-3	Row 7-4	Row 7-5	Row 8-1
Row 8-2	Row 8-3	Row 8-4	Row 8-5	Row 9-1	Row 9-2	Row 9-3	Row 9-4	Row 9-5

Columnar storage

Row-stores

Writing one row at a time is easy for a row-store because it appends the whole row to a chunk of space in storage.

the row oriented database is partitioned horizontally

Are also good for point lookups and index range scans.

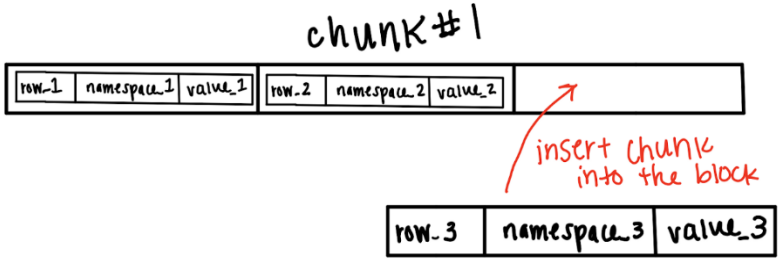
Indexing (creating a key from columns) based on your access patterns can optimize queries and prevent full table scans in a large row-store

Note that creating many indices will create many copies of data

If only one field of the record is desired

using a row-store becomes expensive since all the fields in each record will be read.

Even data that isn't needed for the query response will be read, assuming it isn't indexed properly.



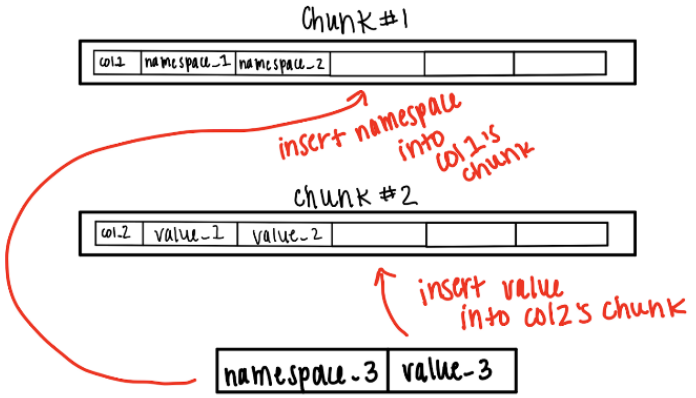
Columnar storage

Consider a different storage for a typical OLAP query

a single column is aggregated across a large portion of the table that happens to include all of the rows shown here.

Columnar storage

- Column-stores
  - Analytical applications often need aggregate data, where only a subset of a table's attributes are needed.
  - Individual values are stored contiguously in storage by column
    - *Column oriented databases are partitioned vertically*
  - The advantage of a columnar-store is that
    - *partial reads are much more efficient*
      - because a lower volume of data is loaded due to reading only the relevant data instead of the whole record.
    - *compression is achieved more efficiently*
      - than in row-stores because columns have uniform types (ex: all strings or integers).



Columnar storage

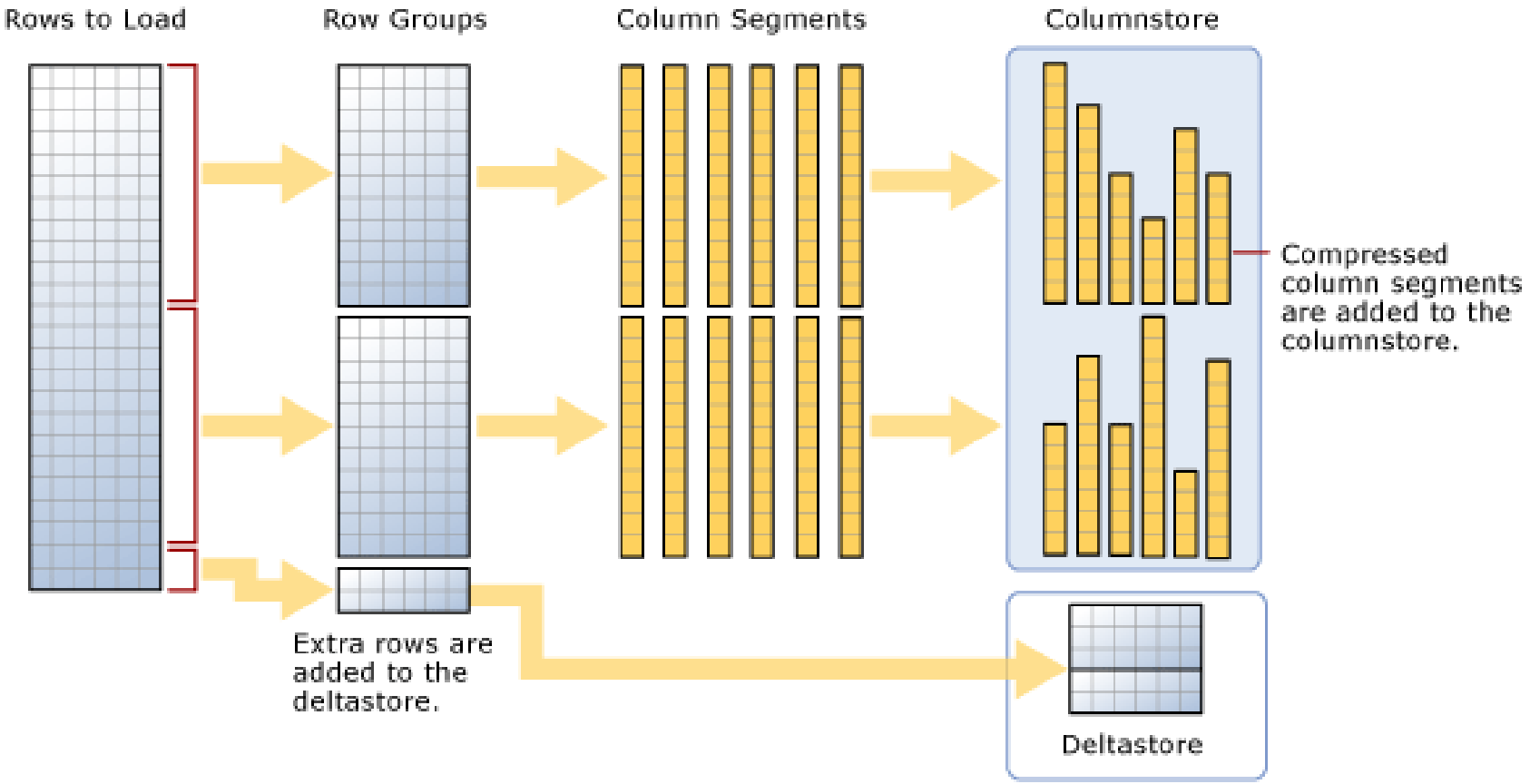
SQL Server Page: Row Data

Row1Col1	Row1Col2	Row1Col3	Row1Col4	Row1Col5
Row1Col6	Row2Col1	Row2Col2	Row2Col3	Row2Col4
Row2Col5	Row2Col6	Row3Col1	Row3Col2	Row3Col3
Row3Col4	Row3Col5	Row3Col6	Row4Col1	Row4Col2
Row4Col3	Row4Col4	Row4Col5	Row4Col6	Row5Col1
Row5Col2	Row5Col3	Row5Col4	Row5Col5	Row5Col6
Row6Col1	Row6Col2	Row6Col3	Row6Col4	Row6Col5
Row6Col6	Row7Col1	Row7Col2	Row7Col3	Row7Col4

SQL Server Page: Column Data

Row1Col1	Row2Col1	Row3Col1	Row4Col1	Row5Col1
Row6Col1	Row7Col1	Row8Col1	Row9Col1	Row10Col1
Row11Col1	Row12Col1	Row13Col1	Row14Col1	Row15Col1
Row16Col1	Row17Col1	Row18Col1	Row19Col1	Row20Col1
Row21Col1	Row22Col1	Row23Col1	Row24Col1	Row25Col1
Row26Col1	Row27Col1	Row28Col1	Row29Col1	Row30Col1
Row31Col1	Row32Col1	Row33Col1	Row34Col1	Row35Col1
Row36Col1	Row37Col1	Row38Col1	Row39Col1	Row40Col1

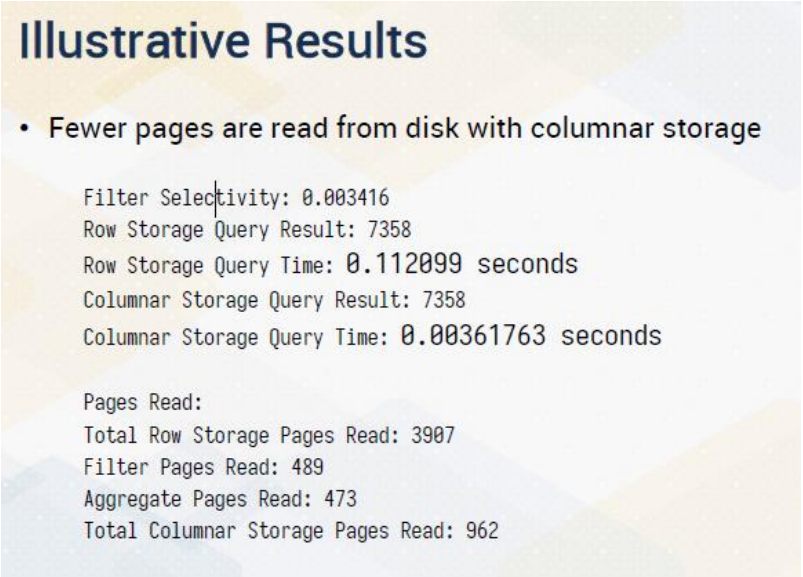
Hybrid


Rows to Load
Row Groups
Column Segments
Columnstore
Compressed column segments are added to the columnstore.
Extra rows are added to the deltastore.
Deltastore

Columnar storage

Example

- Schema stores four columns in a table with 1 million rows.
  - *Timestamp*
  - *Temperature*
  - *Humidity*
  - *Wind speed*
- Randomly generated data
- Page-based Storage:
  - *Organizes data in 4096-byte pages to optimize read/write efficiency.*



Column-oriented databases allow for better data compression ratios compared to row-oriented databases.

- *This is because compression algorithms can take advantage of the similar or repetitive values stored in each column.*
  - For example, dictionary encoding can be applied to store unique values in a dictionary, replacing the original values with shorter references.
- *Common algorithms*
  - Bit-Packing (small-range numeric values)
  - Huffman Coding (frequent categorical strings)
  - Byte Dictionary (repeated values, fewer than 256)

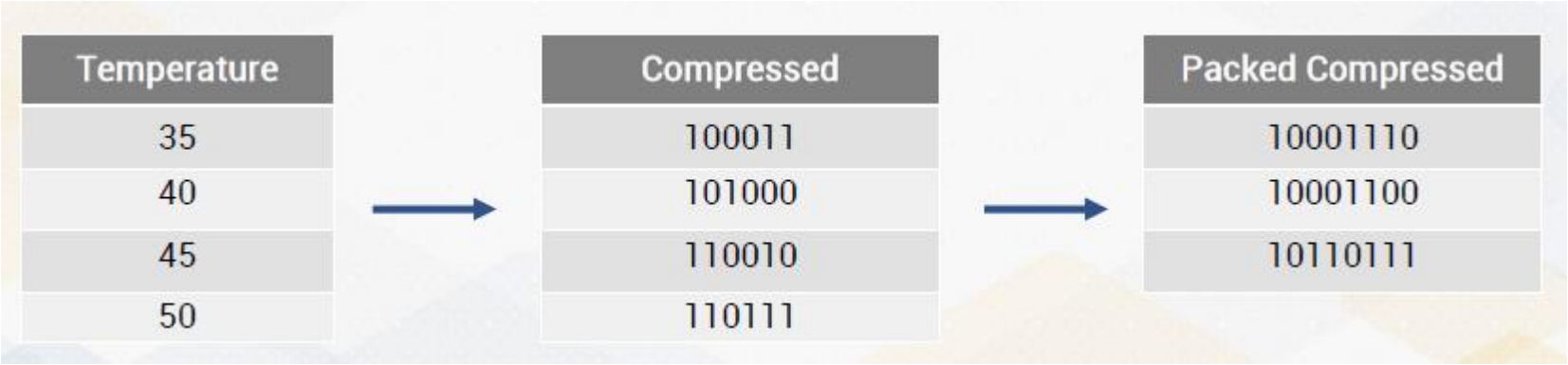
Bit Packing

Reduce storage by only using the minimum number of bits needed to represent values.

Suitable for columns with small ranges.

Example:

If temperatures are between 0 and 15, only 4 bits are needed instead of 32 (int)



Temperature	Compressed	Packed Compressed
35	100011	10001110
40	101000	10001100
45	110010	10110111
50	110111	

Huffman Coding

Assigns shorter codes to frequently occurring values and longer codes to less frequent ones.

Suitable for columns with high-frequency categorical data, like product IDs.

Example: Product IDs that frequently appear in a sales table can be represented with shorter binary codes.

Shorter codes are assigned to more frequent IDs, Longer codes are assigned to less frequent IDs

Product ID	Frequency
111111	100
222222	60
333333	30
444444	10

Compressed
0
10
110
111

Byte Dictionary Encoding

- Assigns unique byte values (0-255) to frequently occurring strings or values, storing each as a single byte.
- Efficient for columns with many repeated values, such as product IDs or categories.
  - *Example: Each product ID is assigned a unique byte, reducing storage size for frequent IDs.*
- Reverse Dictionary = each byte value in encoded_data maps back to the original product ID using reverse_dict

Illustrative Results

```
Average Temperature (Row Storage)     : 36.9554°C
Query Time (Row Storage)              : 17.57 milliseconds
Query Time (Columnar Storage)         :  5.13 milliseconds
Query time on compressed data         :  4.08 milliseconds
```

Illustrative Results

```
Total quantity sold: 412712, Total sales: $2.26383e+07
Uncompressed query time              : 7.79113 milliseconds
Huffman compressed query time        : 2.24537 milliseconds
Byte Dictionary compressed query time: 0.707792 milliseconds
```

Vectorized Processing

standard for speeding up data processing on modern hardware for building highly efficient analytical query engines

requires columnar representation of the data to write highly optimized query processing algorithms

significant departure from conventional tuple-based query processing model

requires a rewrite if query processing algorithms to work on columns

makes efficient utilization of CPU cache

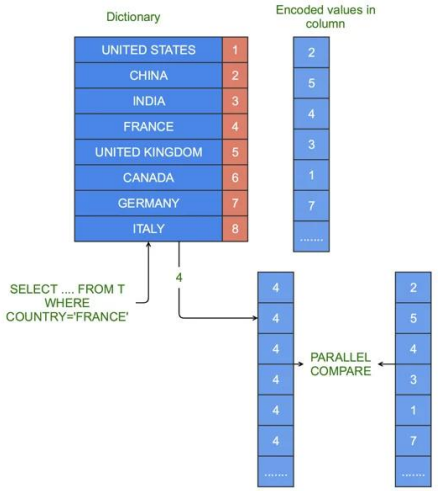
instead of pushing one tuple at a time up the query plan tree, you basically push a block

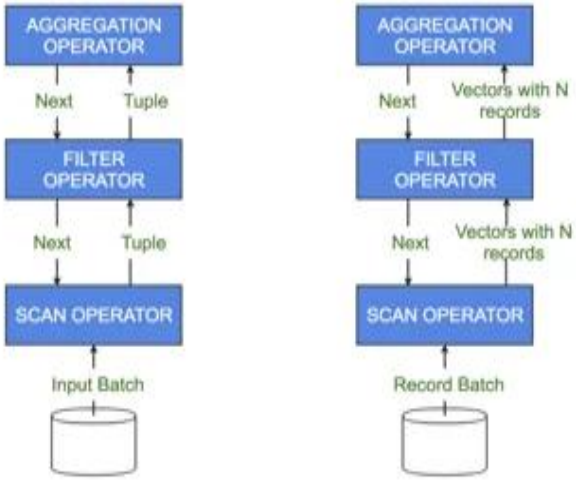
difference between the vectorized model and the traditional model

block comprises of fixed set of tuples (records) represented by a set of vectors with one-to-one correspondence between vector and column/field in the schema

modern processors have extended instruction sets that extend the concept of vectorized execution to multiple column values in parallel in a single instruction – Single Instruction Multiple Data (SIMD)

SIMD exploits data level parallelism independent of concurrency.





Columnar storage

Star schemas work extremely well in columnar databases.

- Better than 3NF designs.

To effectively use columnar storage in a star schema data warehouse, combine the structural advantages of the star schema with the performance benefits of columnar databases.

- Optimize Queries:
  - *Use column-specific selections, filters, and aggregations*
- Write Overhead:
  - *Columnar storage is slower for row-wise inserts. Batch writes and use micro-batching in ETL pipelines*
- Storage Redundancy:
  - *Star schemas duplicate dimension keys in fact tables, but columnar compression mitigates storage costs*
- Hybrid Approaches - Wide Tables for Reporting:
  - *Create denormalized "One Big Table" (OBT) layers atop the star schema for self-service analytics, reducing join complexity*
    - Use columnar storage for these tables to maintain query performance
  - *But, because columnar databases have efficient compression and a lot of advantages for locating data in large tables, doesn't mean you throw data modeling best practices out the window.*
    - You do not gain anything by implementing 'big flat tables' other than, maybe, shaving a few milliseconds off the query time.
    - Data modelling is mainly about providing organization and understanding to the data.

Parquet

Parquet file is an example of hybrid structure:

Row Groups: Horizontal split into logical blocks (e.g. 500 000 rows).

Column Chunks: Inside Row Groups data is spilt vertically into columns.

Pages: Smallest write unit (around 1 MB).

Here is the real encoding and compression, here the SQL engine is working on the data

File Footer: Contains schema information and location (addresses) of all column and row groups.

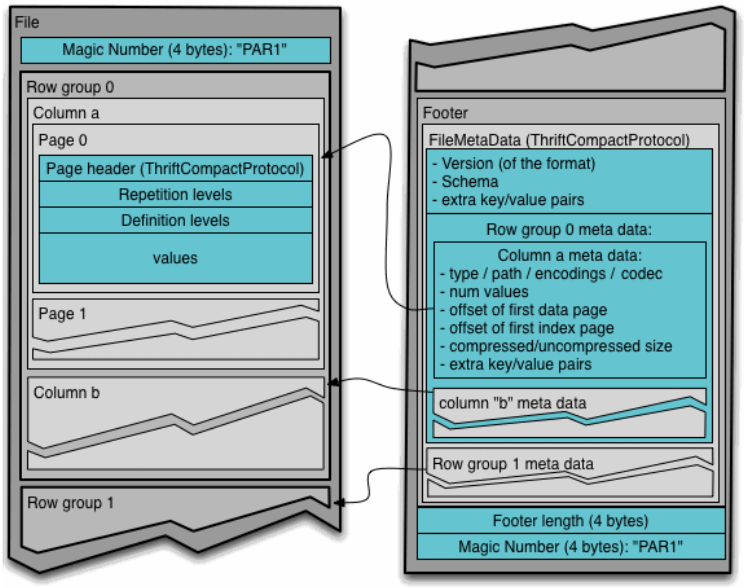
Typically contains also basic statistics, like min/max, null counts, for each page and row group.

Unit of parallelization:

MapReduce - File/Row Group

IO - Column chunk

Encoding/Compression - Page



Parquet

Encoding techniques:

Applied before Parquet compresses data (Snappy)

Dictionary Encoding:

Infile dictionary

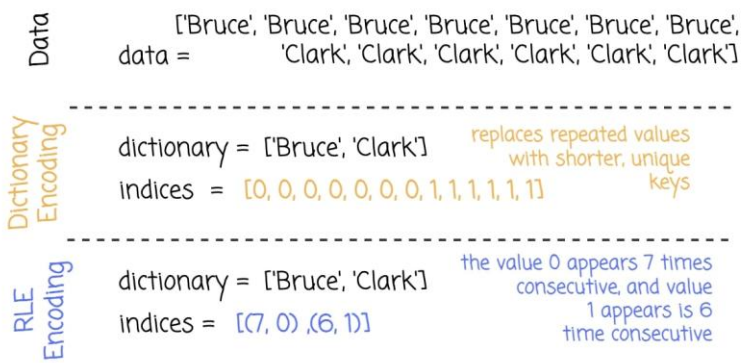
Run-Length Encoding (RLE):

If data, after sorting, have a tendency to form long sequences

Bit Packing:

Knowing max/min values we can apply better representation

For instance to store values 0-3, we can use 2 bits and not full blown 32bit Integer.



Open-table formats

Parquet has one main fundamental flaw:

- ***it is a single file format***

Typically, we have table(s) comprised of many different Parquet files

- *How to manage such a structure*
- *How to optimize their usage*

In order to solve this problem **Open-Table Formats** were introduced

- *Provide programable metadata layer, which hides before the user the fact that a collection of Parquet (or other) files are just one consistent table*
  - Provide SQL access, ACID, etc.
- *Tabular formats handle this issue by introducing a central log*
  - **Transaction Log / Metadata Layer**

Open-table formats

Three main solutions are currently available on the market:

- **Delta Lake (Databricks)**
  - *Works based on a centralized transaction log (Transaction Log) located in the _delta_log/ folder.*
  - *Every write operation creates a JSON file (e.g., 0001.json), which states: "From now on, files A and B belong to the table, and file C is removed."*
  - ***Advantages:*** *Exceptionally fast within the Apache Spark/Databricks ecosystem.*
- **Apache Iceberg (Netflix)**
  - *Iceberg takes a more hierarchical approach and relies on a so-called "metadata tree" – it defines a table through a tree of manifest files (text files describing other files), thereby not utilizing a folder structure.*
  - ***Advantages:*** *It handles petabyte-scale tables because the engine does not need to scan the directory structure in the cloud (so-called object storage file listing).*
- **Apache Hudi (Uber)**
  - *Designed specifically for real-time data streaming and frequent updates (low-latency streaming).*
  - *Offers advanced mechanisms such as Merge-on-Read (updates are saved in small side-files and merged with the main Parquet file only when the table is read by the user).*

Modern ROLAP

Modern storage architecture

Logical layer (Open-Tables - Delta/Iceberg):

Guarantees ACID, allows for Time Travel and makes sure that the BI system sees a consistent view of the data

File layer (Apache Parquet):

Responsible for ultraefficient arrangement of data into columns, their proper encoding and micro-indexing

Physical layer (Cloud Object Storage):

Cheap cloud storage, which physically stores data files

Or at least wants to – as underneath there is a pure key-value store

As such maintaining hierarchical structures of directories is not that easy

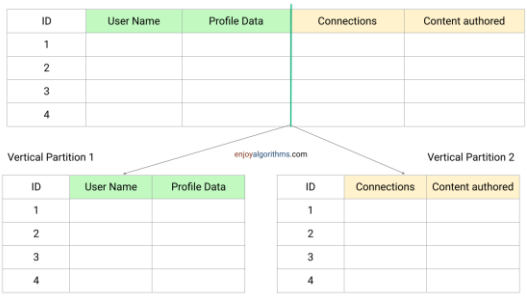
Partitioning

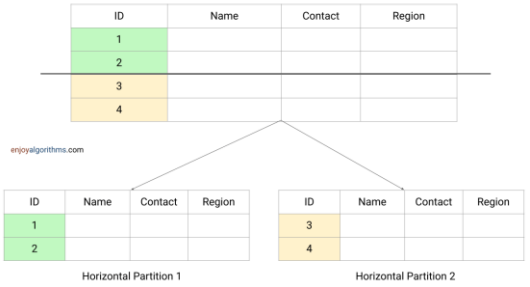
Partitioning or fragmentation

- *divides a table into smaller data sets to better support the management and processing of very large volumes of data*
  - each one called a partition.
- *can be applied to tables as well as to indexes.*
  - a partitioned index can be defined over an unpartitioned table
  - a partitioned table may have unpartitioned indexes defined over it.

There are two ways of partitioning a table: vertically and horizontally.

- Vertical partitioning
  - *splits the attributes of a table into groups that can be independently stored.*
  - *a table can be partitioned such that the most often used attributes are stored in one partition, while other less often used attributes are kept in another partition.*
  - *more records can be brought into main memory, reducing their processing time.*
- Horizontal partitioning
  - *also known as database sharding*
  - *divides a table into smaller tables that have the same structure than the full table but fewer records.*
  - *if some queries require the most recent data while others access older data, a fact table can be horizontally partitioned according to some time frame*
    - advantage is that refreshing the data warehouse is more efficient since only the last partition must be accessed.





Partitioning into smaller data sets

facilitates administrative tasks

increases query performance

especially when parallel processing is applied

enables access to a smaller subset of the data

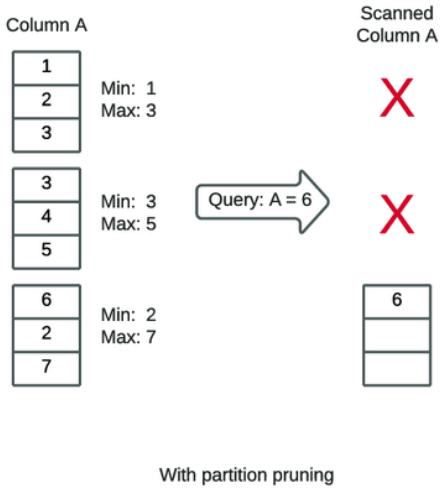
if the user's selection does not refer to all partitions

is recommended when the contents of a table need to be distributed across different servers

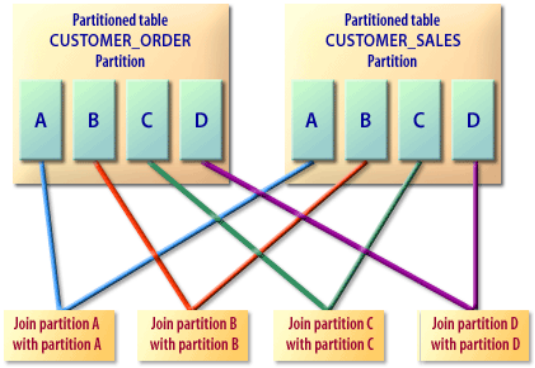
advised in order to perform maintenance on parts of the data without invalidating the entire index

There are two classic techniques of partitioning related to query evaluation.

- Partition pruning
  - *is the typical way of improving query performance using partitioning, often producing performance enhancements of several orders of magnitude.*
- Partition-Wise Joins
  - *The execution time of joins can also be enhanced by using partitioning.*
  - *break a large join into smaller joins that occur between each of the partitions*
    - offers significant performance benefits both for serial and parallel execution.
  - *This occurs when the two tables to be joined are partitioned on the join attributes or, in the case of foreign key joins, when the reference table is partitioned on its primary key.*

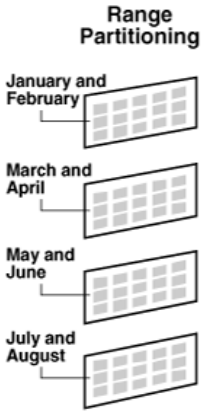


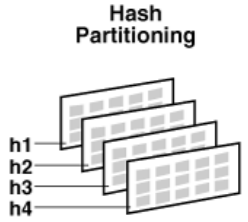
With partition pruning

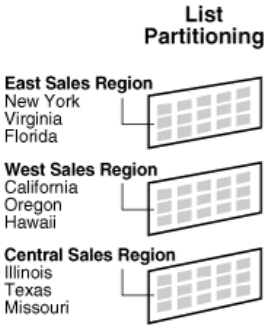


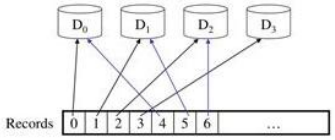
There are three most common partitioning strategies in database systems:

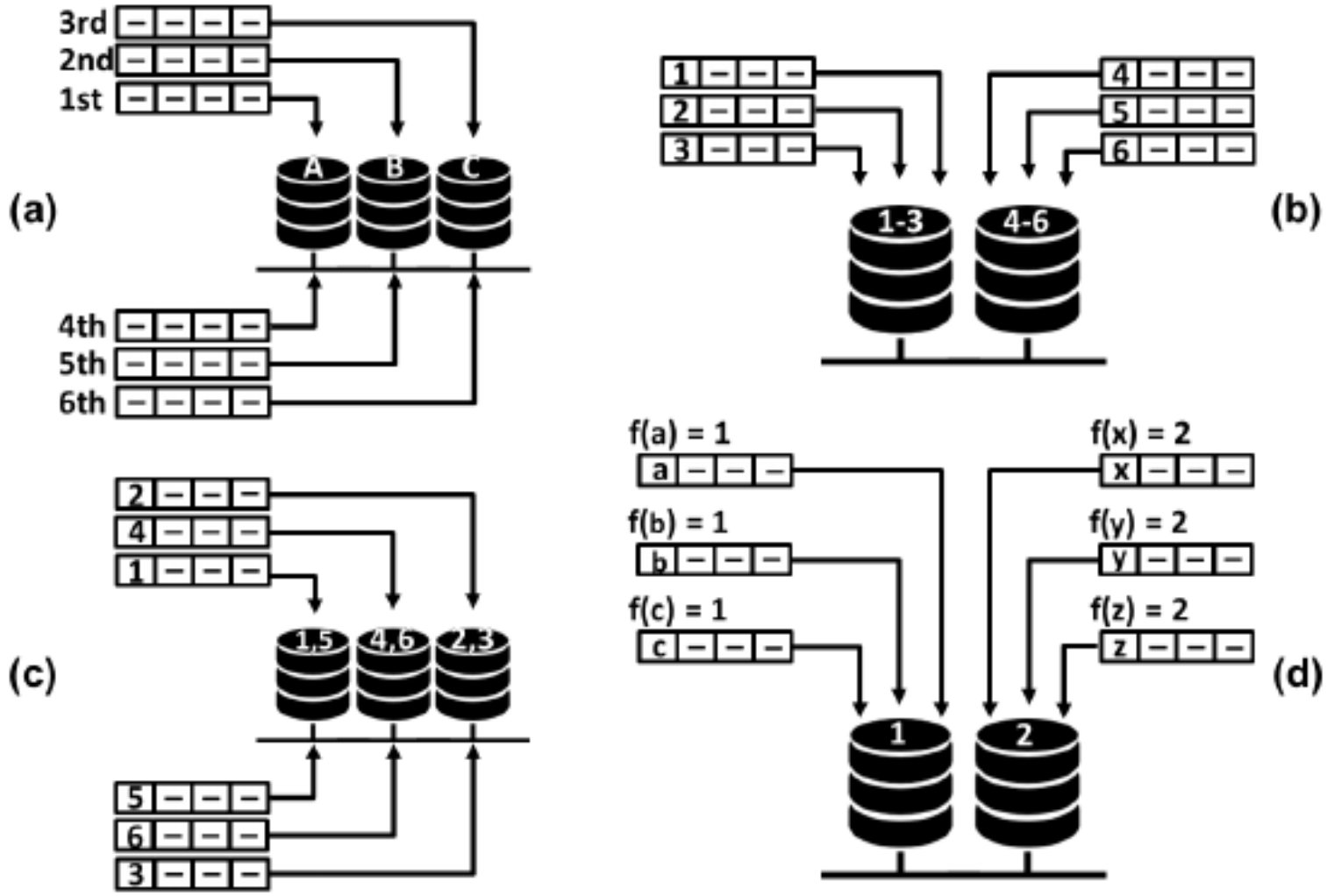
- Range partitioning
  - *which maps records to partitions based on ranges of values of the partitioning key.*
  - *The temporal dimension is a natural candidate for range partitioning, although other attributes can be used.*
    - For example, if a table contains a date column defined as the partitioning key, the January 2012 partition will contain rows with key values from January 1, 2012, to January 31, 2012.
- Hash partitioning
  - *maps records to partitions based on a hashing algorithm applied to the partitioning key*
  - *The hashing algorithm distributes rows among partitions in a uniform fashion, yielding, ideally, partitions of the same size.*
    - This is typically used when partitions are distributed in several devices and, in general, when data are not partitioned based on time since it is more likely to yield even record distribution across partitions.
- List partitioning
  - *enables to explicitly control how rows are mapped to partitions specifying a list of values for the partitioning key.*
    - In this way, data can be organized in an ad hoc fashion.
- Round-robin
  - *Data is cyclically and equally distributed among partitions – regardless of the properties of the data, each split is sequentially assigned the next accessible data item.*
- Composite partitioning
  - *Partitioning based on two or more partitioning techniques, e.g., range-range, range-hash.*










3rd
2nd
1st
A
B
C
(a)
4th
5th
6th
1
2
3
4
5
6
1-3
4-6
(b)
2
4
1
1,5
4,6
2,3
(c)
5
6
3
f(a) = 1
a
f(b) = 1
b
f(c) = 1
c
f(x) = 2
x
f(y) = 2
y
f(z) = 2
z
1
2
(d)

Partitioning Technique	Description	Suitable Data	Query Performance	Data Distribution	Complexity
Horizontal Partitioning	Divides dataset based on rows/ records	Large datasets	Complex joins	Uneven distribution	Distributed transaction management
Vertical Partitioning	Divides dataset based on columns/ attributes	Wide tables	Improved retrieval	Efficient storage	Increased query complexity
Key-based Partitioning	Divides dataset based on specific key	Key-value datasets	Efficient key lookups	Even distribution by key	Limited query flexibility
Range Partitioning	Divides dataset based on specific range	Ordered datasets	Efficient range queries	Even distribution by range	Joins and range queries
Hash-based Partitioning	Divides dataset based on hash function	Unordered datasets	Even distribution	Random distribution	Inefficient key-based queries
Round-robin Partitioning	Divides dataset in a cyclic manner	Equal-sized datasets	Basic load balancing	Even distribution	Limited query optimization

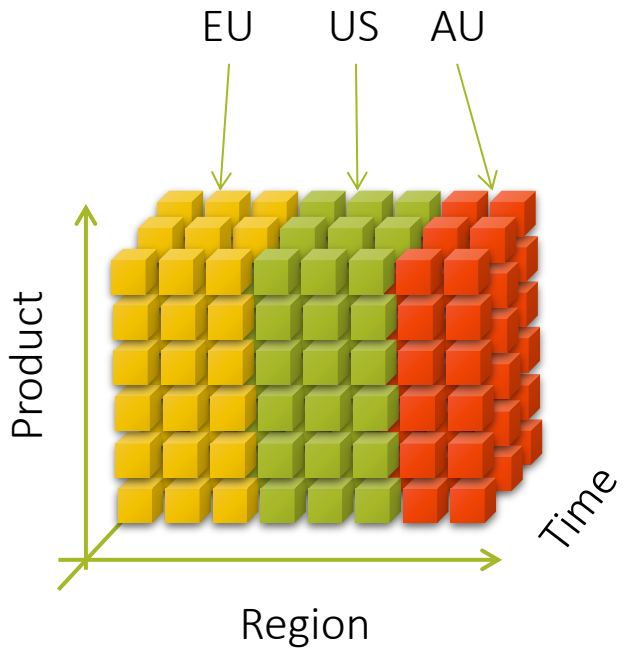
MOLAP

- Partitioned Cubes
  - To overcome the space limitation of MOLAP, the cube can be partitioned
    - *One logical cube of data can be spread across multiple physical cubes on separate (or same) servers*
      - The divide&conquer cube partitioning approach helps alleviate the scalability limitations of MOLAP implementation
  - Ideal cube partitioning is completely invisible to end users
  - Performance degradation does occur in case of a join across partitioned cubes

MOLAP

Advanced MOLAP implementations

Allow to distribute queries across partitioned cubes (potentially on distinct server platforms) for parallel retrieval



Bibliography

SOURCES

Jensen C.S., Pedersen T.B., Thomsen C.,

Multidimensional Databases and Data Warehousing,

Morgan & Claypool Publishers series SYNTHESIS LECTURES ON DATA MANAGEMENT, 2010

Inmon W.,

Building the Data Warehouse,

John Wiley & Sons, New York 2002

Sherman R.,

Business Intelligence Guidebook

Elsevier, 2015

Claudia Imhoff, Nicholas Galemmo, Jonathan G. Geiger,

Mastering Data Warehouse Design - Relational and Dimensional Techniques,

Wiley Publishing, Inc., 2003

Efraim Turban, Ramesh Sharda, Dursun Delen, David King, Janine E. Aronson

Business Intelligence: A Managerial Approach

Prentice Hall, 2010

Sanjiv Purba (ed.)

Handbook of Data Management (1999 Edition)

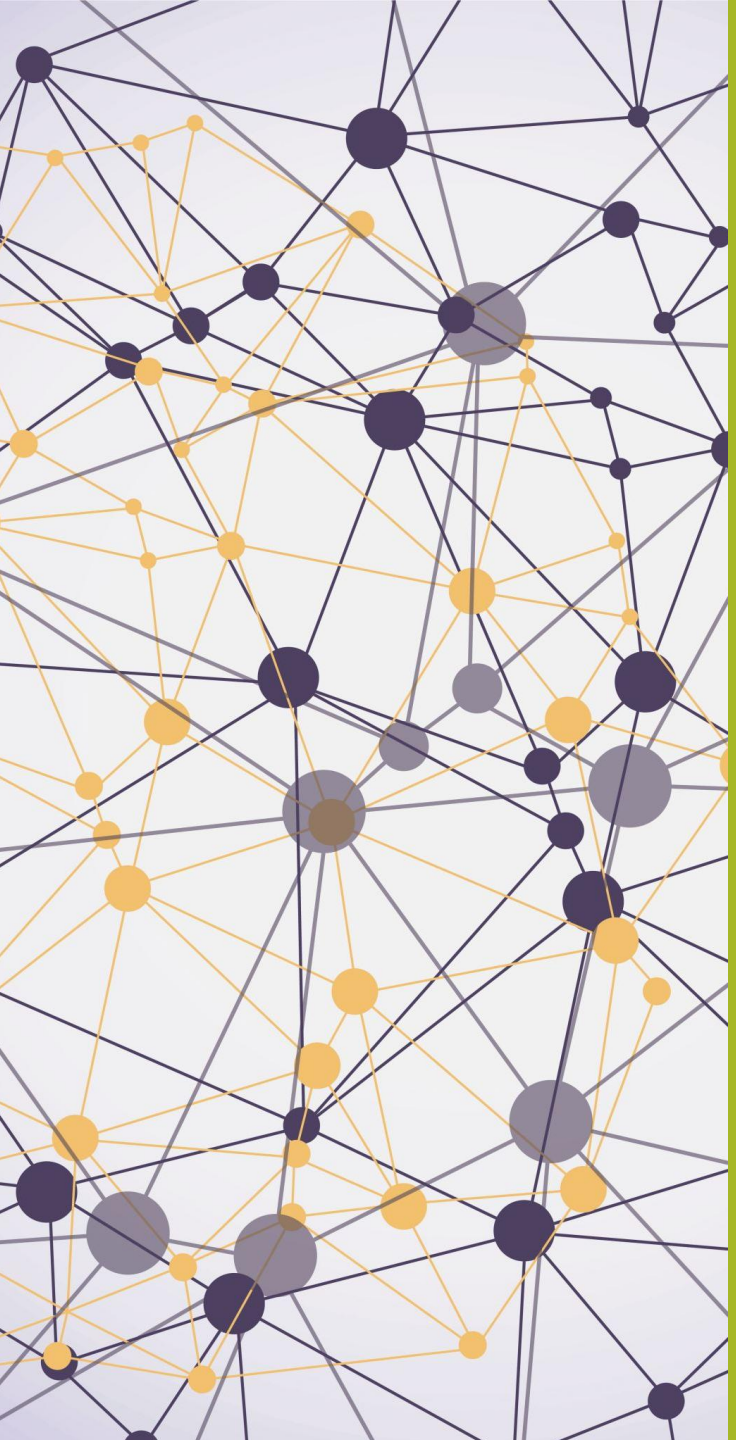
Auerbach, 1999

Joy Arulraj

Lecture for Database System Implementation, 2024

Georgia Tech, College of Computing

https://faculty.cc.gatech.edu/~jarulraj/courses/4420-f24/slides/22-columnar-storage-and-compression.pdf



View maintenance

Updating cuboids

In data warehouses, materialized views that include aggregate functions are called **summary tables**

The problem:

- as data at the sources are added or updated, the summary tables that depend on these data must be also updated
  - *summary tables can be maintained with minimum access to the base data while keeping maximum data availability*

Two options:

- to recompute the summary tables from scratch
- to apply incremental view maintenance techniques to avoid such recomputation

View maintenance

under certain conditions, it is possible to compute changes to the view caused by changes in the underlying relations without recomputing the entire view from scratch.

incremental view maintenance

$\mathsf{Sales}(\mathsf{ProductKey}, \mathsf{CustomerKey}, \mathsf{Quantity})$

$\mathsf{TopProducts} = \pi_{\mathsf{ProductKey}}(\sigma_{\mathsf{Quantity}>150}(\mathsf{Sales}))$

$+ (\mathrm{p2}, \mathrm{c3}, 110)$

$+ (\mathrm{p2}, \mathrm{c3}, 160)$

$- (\mathrm{p2}, \mathrm{c3}, 160)$

requires to scan the relation Sales

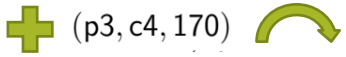
view maintenance

under certain conditions, it is possible to compute changes to the view caused by changes in the underlying relations without recomputing the entire view from scratch.

incremental view maintenance

$\text{Sales}(\text{ProductKey}, \text{CustomerKey}, \text{Quantity})$

$\text{FoodCustomers} = \pi_{\text{CustomerKey}}(\sigma_{\text{CategoryName}='\text{Food}'}(\text{Product}) * \text{Sales})$



we check in the base relations whether or not p3 is in the food category

An aggregate function is **self-maintainable**

- if the new value of the function can be computed solely from the old values and from changes to the base data

Aggregate functions must be distributive in order to be self-maintainable.

- The five classic aggregate functions in SQL are self-maintainable with respect to insertions, but not to deletions.
  - *SUM, AVG, COUNT, MAX, MIN*
- In fact, MAX and MIN are not, and cannot be made, self-maintainable with respect to deletions

The summary-delta algorithm

- a representative summary table maintenance algorithm
- has two main phases called propagate and refresh:
  - *propagate phase can be performed in parallel with data warehouse operations*
  - *only the refresh phase requires taking the warehouse off-line*
- basic idea is to
  - *create in the propagate phase a so-called* ***summary-delta table*** *that stores the net changes to the summary table due to changes in the source data*
  - *during the refresh phase, these changes are applied to the summary table*
- Idea:
  - *processes two sets of deferred changes:*
    - pos-ins: Table storing deferred insertions; Projects inserted tuples with positive values (e.g. +1 for count, +qty for quantity)
    - pos-del: Table storing deferred deletions; Projects deleted tuples with negative values (e.g., -1 for count, -qty for quantity)
  - *later unions both results and aggregates by the same group-by attributes as the summary table*
    - produces net changes for each aggregate function value

```
CREATE VIEW DailySalesSum AS (
        SELECT    ProductKey, TimeKey, SUM(Quantity) AS SumQuantity,
                  COUNT(*) AS Count
        FROM      Sales
        GROUP BY ProductKey, TimeKey )
```

DailySalesSum

In the propagate phase, we define two tables

Δ+(Sales) and Δ-(Sales)

store the insertions and deletions to the fact table,

and a view where the net changes to the summary tables are stored

```
CREATE VIEW SD_DailySalesSum(ProductKey, TimeKey,
            SD_SumQuantity, SD_Count) AS
        WITH Temp AS (
            ( SELECT ProductKey, TimeKey,
                     Quantity AS _Quantity, 1 AS _Count
              FROM   Δ+(Sales) )
            UNION ALL
            ( SELECT ProductKey, TimeKey,
                     -1 * Quantity AS _Quantity, -1 AS _Count
              FROM   Δ−(Sales) ) )
        SELECT ProductKey, TimeKey, SUM(_Quantity), SUM(_Count)
        FROM Temp
        GROUP BY ProductKey, TimeKey
```

DailySalesSum

During the refresh phase, we apply to the summary table the net changes stored in the summary-delta table

```
CREATE VIEW DailySalesSum AS (
        SELECT    ProductKey, TimeKey, SUM(Quantity) AS SumQuantity,
                  COUNT(*) AS Count
        FROM      Sales
        GROUP BY ProductKey, TimeKey )
```

```
For each tuple T in SD_DailySalesSum DO
    IF NOT EXISTS (
            SELECT *
            FROM   DailySalesSum D
            WHERE  T.ProductKey = D.ProductKey AND
                   T.TimeKey = D.TimeKey)
        INSERT T INTO DailySalesSum
    ELSE
        IF EXISTS (
                SELECT *
                FROM   DailySalesSum D
                WHERE  T.ProductKey = D.ProductKey AND
                       T.TimeKey = D.TimeKey AND
                       T.SD_Count + D.Count = 0)
            DELETE T FROM DailySalesSum
        ELSE
            UPDATE DailySalesSum
            SET SumQuantity = SumQuantity + T.SD_SumQuantity,
                Count = Count + T.SD_Count
            WHERE ProductKey = T.ProductKey AND
                TimeKey = T.TimeKey
```

DailySalesSum

Product Key	Time Key	Sum Quantity	Count
p2	t2	100	10
p3	t3	100	20
p4	t4	100	15
p5	t5	100	2
p6	t6	100	1

$\Delta^+(\text{Sales})$

Product Key	Customer Key	Time Key	Quantity
p1	c1	t1	150
p2	c1	t2	200
p2	c2	t2	100
p4	c2	t4	100
p6	c5	t6	200

$\Delta^-(\text{Sales})$

Product Key	Customer Key	Time Key	Quantity
p2	c1	t2	10
p5	c2	t5	10
p6	c5	t6	100

SD_DailySalesSum

Product Key	Time Key	SD_Sum Quantity	SD_Count
p1	t1	150	1
p2	t2	290	1
p4	t4	100	1
p5	t5	-10	-1
p6	t6	100	0

DailySalesSum after update

Product Key	Time Key	Sum Quantity	Count
p1	t1	150	1
p2	t2	390	11
p3	t3	100	20
p4	t4	200	16
p5	t5	90	1
p6	t6	200	1

Product Key	Time Key	Sum Quantity	Count
p2	t2	100	10
p3	t3	100	20
p4	t4	100	15
p5	t5	100	2
p6	t6	100	1

Product Key	Time Key	SD_Sum Quantity	SD_Count
p1	t1	150	1
p2	t2	290	1
p4	t4	100	1
p5	t5	-10	-1
p6	t6	100	0

Product Key	Time Key	Sum Quantity	Count
p1	t1	150	1
p2	t2	390	11
p3	t3	100	20
p4	t4	200	16
p5	t5	90	1
p6	t6	200	1

Product Key	Customer Key	Time Key	Quantity
p1	c1	t1	150
p2	c1	t2	200
p2	c2	t2	100
p4	c2	t4	100
p6	c5	t6	200

Product Key	Customer Key	Time Key	Quantity
p2	c1	t2	10
p5	c2	t5	10
p6	c5	t6	100

DailySalesMax

Product Key	Time Key	Max Quantity	Count
p2	t2	150	4
p3	t3	100	5
p4	t4	50	6
p5	t5	150	5
p6	t6	100	3
p7	t7	100	2

SD_DailySalesMax

Product Key	Time Key	SD_Max Quantity	SD_Count
p1	t1	100	2
p2	t2	100	2
p3	t3	100	2
p4	t4	100	2
p5	t5	100	-2
p6	t6	100	-2
p7	t7	100	-2

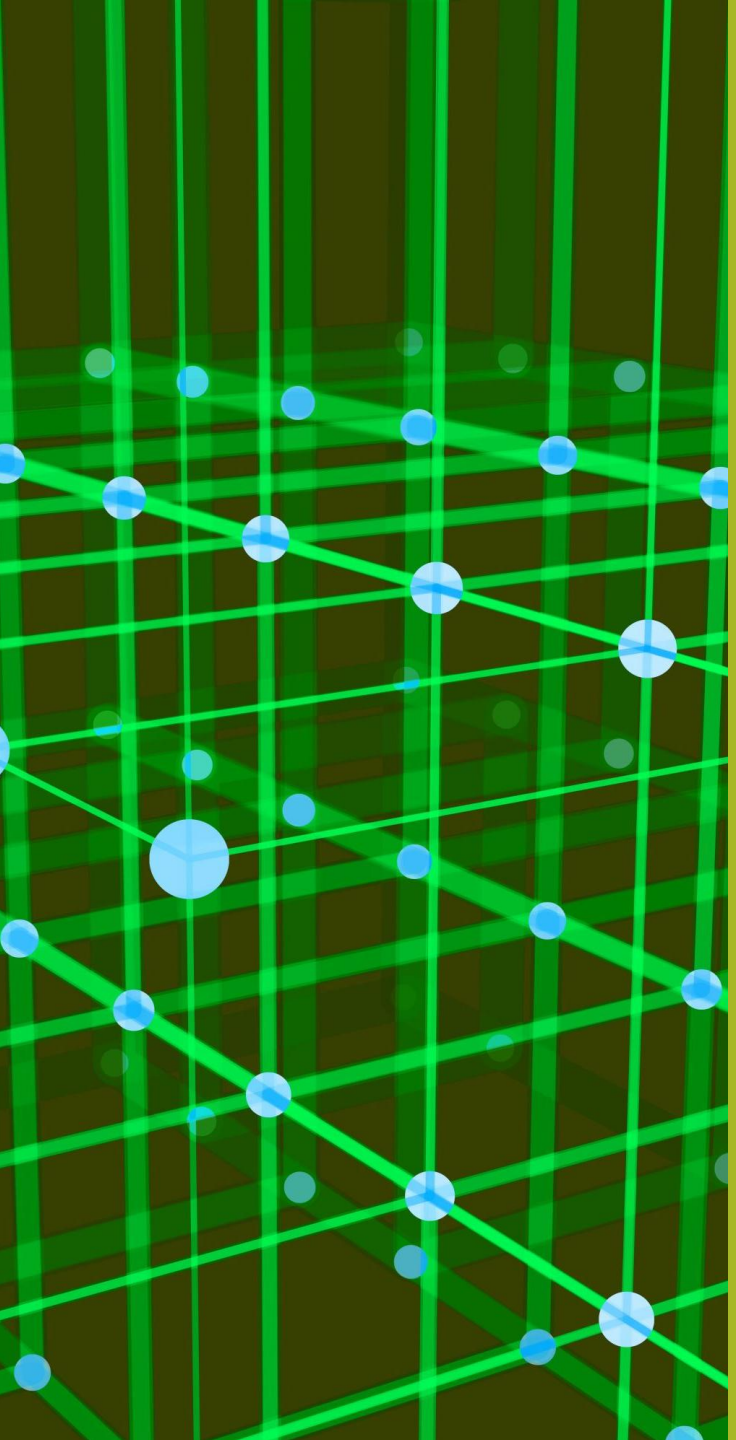
Updated view DailySalesMax

Product Key	Time Key	Max Quantity	Count
p1	t1	100	2
p2	t2	150	4
p3	t3	100	7
p4	t4	100	2
p5	t5	150	5
p6	t6	100	1
p7	t7	?	?

Product Key	Time Key	Max Quantity	Count
p2	t2	150	4
p3	t3	100	5
p4	t4	50	6
p5	t5	150	5
p6	t6	100	3
p7	t7	100	2

Product Key	Time Key	Max Quantity	Count
p1	t1	100	2
p2	t2	150	4
p3	t3	100	7
p4	t4	100	2
p5	t5	150	5
p6	t6	100	1
p7	t7	?	?

Product Key	Time Key	SD_Max Quantity	SD_Count
p1	t1	100	2
p2	t2	100	2
p3	t3	100	2
p4	t4	100	2
p5	t5	100	-2
p6	t6	100	-2
p7	t7	100	-2



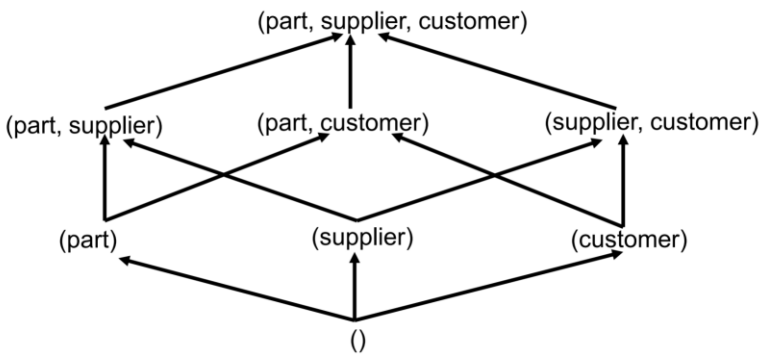
Data Computation

Data Cube Computation Methods

Data cube computation is an essential task in data warehouse implementation.

- The precomputation of all or part of a data cube can greatly reduce the response time and enhance the performance of online analytical processing.

However, such computation is challenging because it may require substantial computational time and storage space.



Partial order

Q1=(part,supplier, *) can be derived based on Q2=(part, supplier, customer)

Q1<Q2

Also for hierarchies

(part,country,) < (part,city,*)*

With a constrained space of 2^S (the power space of a set of views), what is the optimal choice of views to materialize?

Cost model

Estimating the benefits of materializing a view – related to the evaluation strategy

It is often impossible to know the sizes of views without actually calculating them

Data:

for each view, its size, the number of k

Search:

k views with the highest profit

It is a NP-Complete problem

The problem of optimally selecting cuboids to materialize in a data warehouse can be reduced to the **set cover problem**

Algorithms like Bottom-Up Computation and Star-Cubing achieve O(nlogn) for specific cases

Ashadevi. B., Analysis of View Selection Problem in Data Warehousing Environment,
International Journal of Engineering and Technology Vol.3 (6), Engg Journals Publications, 2011-2012, pp. 447-457

Several algorithmic strategies have been developed to address the view materialization optimization problem:

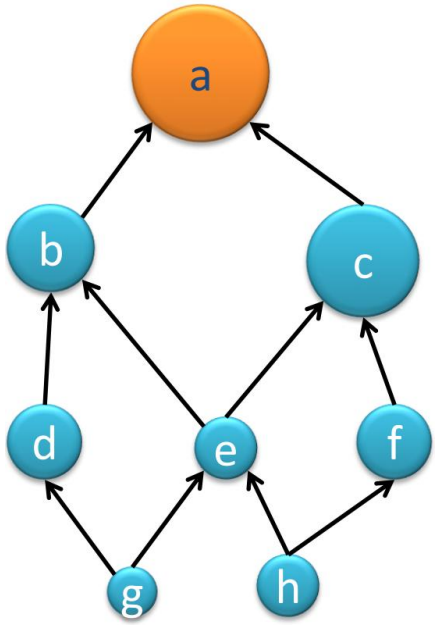
- Greedy Algorithms:
  - *These approaches iteratively select views that provide the greatest benefit-to-cost ratio, considering factors such as query frequency, view size, and maintenance requirements.*
  - *The greedy method incorporates maintenance costs and storage constraints while utilizing "And-Or" view graphs to represent all possible ways to generate warehouse views.*
- Heuristic Algorithms:
  - *These methods utilize Multiple View Processing Plans (MVPP) to obtain optimal materialized view selections that achieve the best combination of performance and low maintenance cost.*
  - *Heuristic approaches can provide feasible solutions based on individual optimal query plans when exact optimization becomes computationally prohibitive.*
- Genetic Algorithms:
  - *Advanced evolutionary approaches have been developed that use binary encoding where 0 indicates a view is not materialized and 1 indicates materialization.*
  - *These algorithms have shown superior performance compared to conventional heuristic methods in terms of both speed and accuracy.*

Selection Criteria and Best Practices for optimal view materialization requires careful consideration of several key factors:

- Query Patterns and Frequency:
  - *Views that support frequently executed queries should receive higher priority for materialization, as the performance benefits will be realized more often.*
  - *Analysis of historical query workloads helps identify the most valuable materialization candidates.*
- Data Filtering and Aggregation:
  - *The most effective materialized views typically perform one or both of two key functions: filtering data (either by rows or columns) and performing resource-intensive operations such as complex aggregations.*
  - *Views that filter to recent data or unusual patterns often provide significant performance benefits.*
- Storage Effectiveness:
  - *The concept of storage effectiveness (η) measures the benefit-to-storage ratio for each potential materialized view.*
  - *Views with high storage effectiveness yield the greatest performance improvement per unit of storage space consumed.*
- Maintenance Windows:
  - *The available time for view maintenance operations constrains which views can be practically materialized.*
  - *Views requiring frequent updates may not be suitable for environments with limited maintenance windows.*

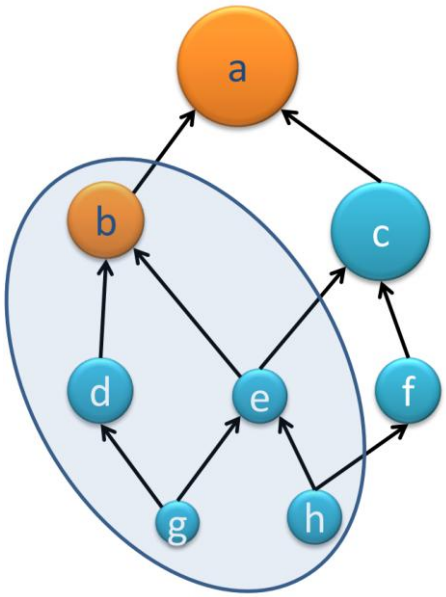
Greedy approach

Query	Size
a	100
b	50
c	75
d	20
e	30
f	40
g	1
h	10



Greedy approach

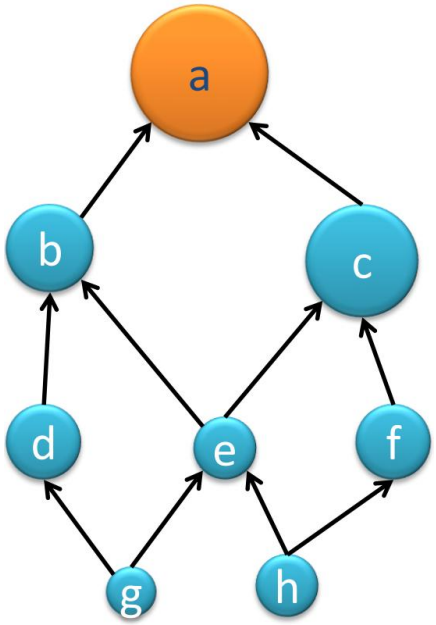
Query	Size
a	100
b	50
c	75
d	20
e	30
f	40
g	1
h	10



Query	Size	Uses?	Cost	Benefit
a	100	a	100	-
b	50	**b**	**50**	**50**
c	75	a	100	-
d	20	**b**	**50**	**50**
e	30	**b**	**50**	**50**
f	40	a	100	-
g	1	**b**	**50**	**50**
h	10	**b**	**50**	**50**

Greedy approach

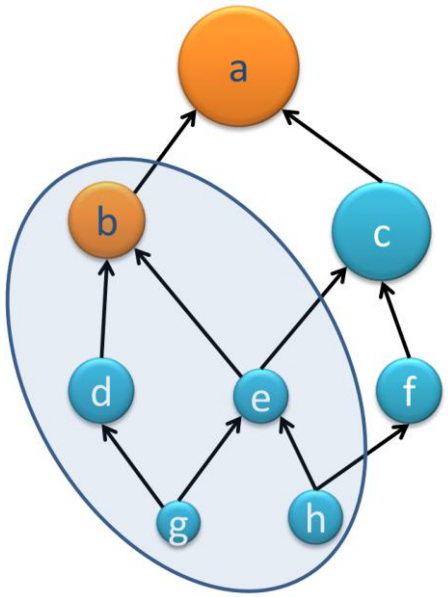
Query	Size
a	100
b	50
c	75
d	20
e	30
f	40
g	1
h	10



Q	b	c	d	e	f	g	h
a							
b	50						
c		25					
d	50		80				
e	50	25		70			
f		25			60		
g	50	25	80	70		99	
h	50	25		70	60		90
Total	250	125	160	210	120	99	90

Greedy approach

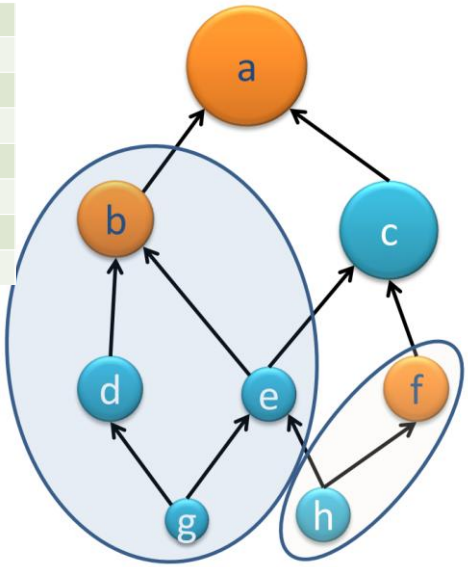
Query	Size
a	100
b	50
c	75
d	20
e	30
f	40
g	1
h	10



Q	c	d	e	f	g	h
a						
b						
c	25					
d		30				
e			20			
f	25			60		
g		30	20		49	
h			20	60		40
Total	50	60	60	70	49	40

Greedy approach

Query	Size
a	100
b	50
c	75
d	20
e	30
f	40
g	1
h	10



Q	c	d	e	g	h
a					
b					
c	25				
d		30			
e			20		
f					
g		30	20	49	
h			20		30
Total	25	60	60	49	30

There are general optimization techniques for efficient computation of data cubes.

- Optimization Technique 1: Sorting, hashing, and grouping.
  - *In cube computation, aggregation is performed on the tuples (or cells) that share the same set of dimension values.*
    - it is important to explore sorting, hashing, and grouping operations to access and group such data together to facilitate computation of such aggregates.
  - *For example,*
    - to compute total sales by branch, day, and item, it can be more efficient to sort tuples or cells by branch, and then by day, and then group them according to the item name.
  - *This technique can also be further extended to perform*
    - shared-sorts (i.e., sharing sorting costs across multiple cuboids when sort-based methods are used),
    - shared-partitions (i.e., sharing the partitioning cost across multiple cuboids when hash-based algorithms are used).

There are general optimization techniques for efficient computation of data cubes.

- Optimization Technique 2: Simultaneous aggregation and caching intermediate results
  - *It is efficient to compute higher level aggregates from previously computed lower-level aggregates*
    - rather than from the base fact table
  - *Simultaneous aggregation from cached intermediate computation results may lead to the reduction of expensive disk I/O operations*
  - *For example,*
    - to compute sales by branch, we can use the intermediate results derived from the computation of a lower-level cuboid, such as sales by branch and day.
  - *This technique can be further extended to perform amortized scans*
    - i.e., computing as many cuboids as possible at the same time to amortize disk reads

There are general optimization techniques for efficient computation of data cubes.

Optimization Technique 3: Aggregation from the smallest child, when there exist multiple child cuboids.

When there exist multiple child cuboids

it is usually more efficient to compute the desired parent (i.e., more generalized) cuboid from the smallest, previously computed child cuboid.

For example to compute a sales cuboid – C_branch

when there exist two previously computed cuboids

C_{branch,year} and *C_{branch,item}*

it is obviously more efficient to compute *C_branch* from the former than from the latter

if there are many more distinct items than distinct years

There are general optimization techniques for efficient computation of data cubes.

Optimization Technique 4: The Apriori pruning method can be explored to compute iceberg cubes efficiently.

The Apriori property, in the context of data cubes, states as follows: If a given cell does not satisfy minimum support, then no descendant of the cell (i.e., more specialized cell) will satisfy minimum support either

can be used to substantially reduce the computation of iceberg cubes.

A common iceberg condition is that the cells must satisfy a minimum support threshold, such as a minimum count or sum

the Apriori property can be used to prune away the exploration of the descendants of the cell

if a condition is violated for some cell c, then every descendant of c will also violate that condition

For example, if the count of a cell, c, in a cuboid is less than a minimum support threshold, v, then the count of any of c's descendant cells in the lower-level cuboids can never be greater than or equal to v, and thus can be pruned.

Computing cubes:

- 3 methodologies
- Bottom-Up:
  - *Multi-Way array aggregation (Zhao, Deshpande & Naughton, SIGMOD'97)*
- Top-down:
  - *Bottom-up cube computation – BUC (Beyer & Ramarkrishnan, SIGMOD'99)*
  - *H-cubing technique (Han, Pei, Dong & Wang: SIGMOD'01)*
- Integrating Top-Down and Bottom-Up:
  - *Star-cubing algorithm (Xin, Han, Li & Wah: VLDB'03)*
- High-dimensional OLAP:
  - *A Minimal Cubing Approach (Li, et al. VLDB'04)*